

Los eventos del sistema pueden incluir parámetros.

El ejemplo adjunto se refiere al curso normal de los eventos en el caso *Comprar Productos*. Indica que el cajero es el único actor en el sistema TPDV y que genera los eventos del sistema *introducirProducto*, *terminarVenta* y *efectuarPago*.

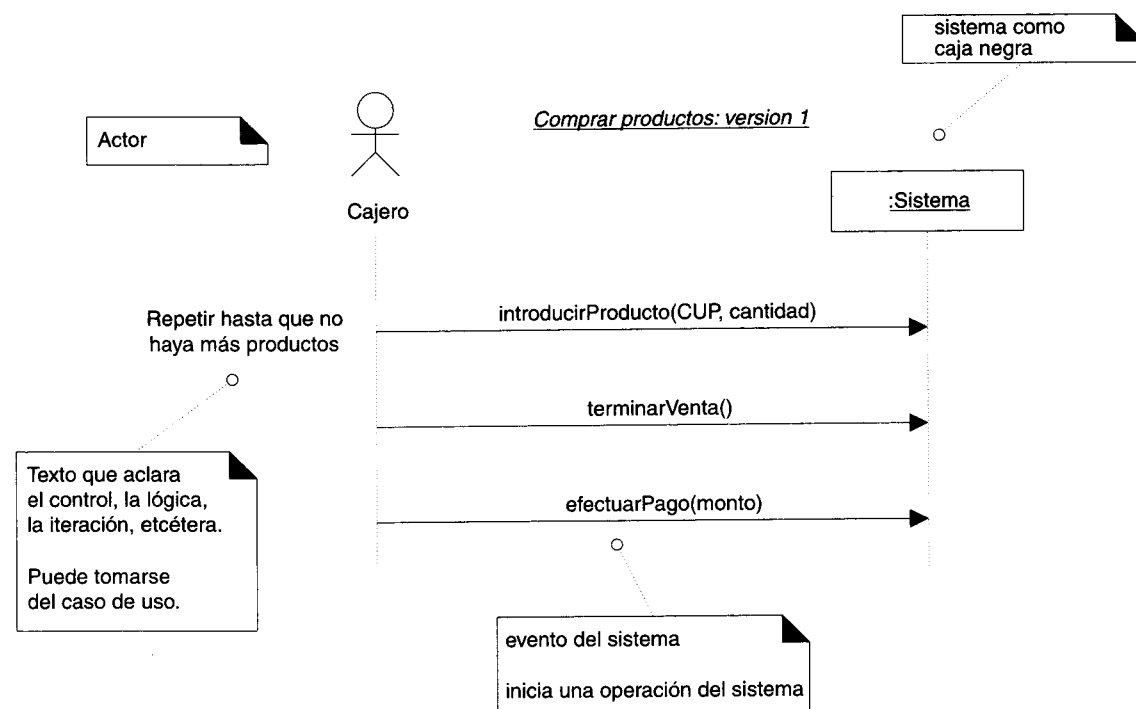


Figura 13.1 Diagrama de la secuencia del sistema para el caso de uso *Comprar productos*.

13.6 Eventos y operaciones de un sistema

El **evento de un sistema** es un hecho externo de entrada que un actor produce en un sistema. El evento da origen a una operación de respuesta. La **operación de un sistema** es una acción que éste ejecuta en respuesta a un evento del sistema (figura 13.2). Por ejemplo, cuando el cajero genera el evento *introducirProducto*, causa la ejecución de la operación *introducirProducto*; el nombre del evento y de la operación son idénticos; la distinción reside en que el evento es el estímulo nombrado y la operación es la respuesta.¹

¹ Lo mismo sucede con los mensajes y métodos.

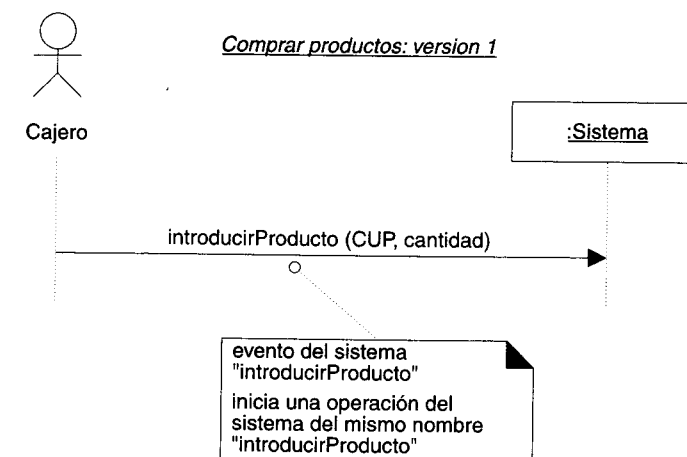


Figura 13.2 Los eventos del sistema inician las operaciones de él.

13.6.1 Registro de las operaciones de un sistema

Para determinar el conjunto de las operaciones requeridas del sistema se identifican sus eventos. Cuando se utilizan parámetros, las operaciones son las siguientes:

- *introducirProducto (CUP, cantidad)*
- *terminarVenta ()*
- *efectuarPago (monto)*

¿Dónde deberían registrarse estas operaciones? El lenguaje UML ofrece una notación para registrar las operaciones de un tipo, como se aprecia en la figura 13.3.



Figura 13.3 Notación de las operaciones en el UML.

Con esta notación, las operaciones del sistema pueden agruparse como operaciones del tipo *Sistema* (figura 13.4). Los parámetros son opcionales: pueden incluirse o no.

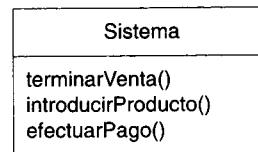


Figura 13.4 Operaciones de un sistema registradas en el tipo *Sistema*.

Este esquema también funciona satisfactoriamente cuando registra las operaciones del sistema de varios sistemas o procesos en una aplicación distribuida; a cada sistema se le asigna un nombre especial (*Sistema1*, *Sistema2*, ...) y también sus operaciones propias.

Observe que la representación del tipo *Sistema* es muy diferente a lo que se expresó en el modelo conceptual. Los elementos de éste representan conceptos del mundo real; en cambio, el tipo *Sistema* es un concepto artificial. Y además muestra las operaciones, cosa totalmente nueva. Ello se debe a que —a diferencia del modelo conceptual, que presenta información estática— estamos describiendo el comportamiento del sistema, que es información dinámica.

13.7 Cómo elaborar un diagrama de la secuencia de un sistema

Para elaborar diagramas de la secuencia de un sistema que describan el curso normal de los eventos en un caso de uso:

1. Trace una línea que represente el sistema como una caja negra.
2. Identifique los actores que operan directamente sobre el sistema. Trace una línea para cada uno de ellos.
3. A partir del curso normal de los eventos del caso de uso identifique los eventos ("externos") del sistema que son generados por los actores. Muéstrellos gráficamente en el diagrama.
4. A la izquierda del diagrama puede incluir o no el texto del caso de uso.

13.8 Diagramas de la secuencia de un sistema y otros artefactos

La identificación y ordenamiento de los eventos de un sistema para incluirlos en los diagramas de secuencia (figura 13.5) se logran de la revisión de los casos de uso.

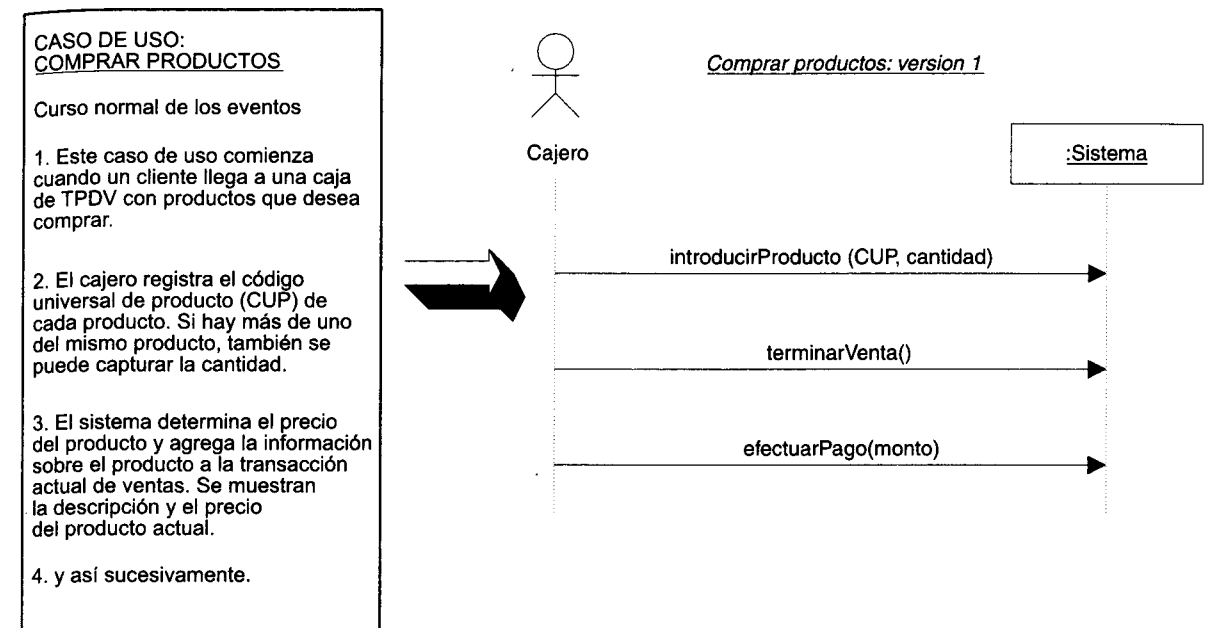


Figura 13.5 Los diagramas de la secuencia de un sistema se dedujeron de los casos de uso.

13.9 Eventos y fronteras de un sistema

Para identificar los eventos de un sistema es necesario tener una idea clara de lo que se quiere al escoger su frontera, según vimos en el capítulo anterior dedicado a los casos de uso. Por lo que respecta al desarrollo del software, la frontera de un sistema suele seleccionarse para que sea el sistema de software (y, posiblemente, del hardware); dentro de este contexto, un evento del sistema es un hecho externo que estimula directamente el software (figura 13.6). En la reingeniería de empresas, la frontera del sistema —y, por tanto, sus eventos— pueden ampliarse para que abarquen los procesos manuales, pero esto rebasa el ámbito de nuestro libro.

Consideremos ahora el caso de uso *Comprar Productos* a fin de identificar los eventos del sistema. Primero debemos determinar los actores que interactúan directamente con el sistema de software. El cliente interactúa con el cajero, pero no directamente con el software TPDV; esto sólo lo hace el cajero. Por tanto, el cliente no es un generador de eventos del sistema; sólo el cajero lo es.

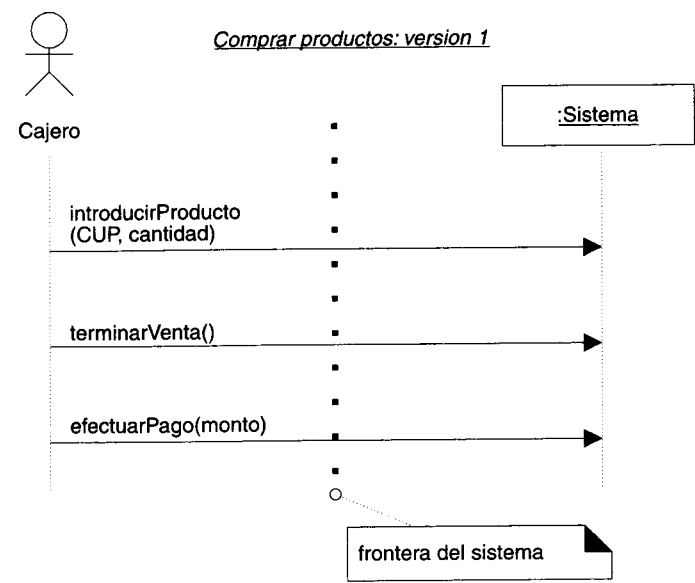


Figura 13.6 Definición de la frontera del sistema.

3.10 Asignación de nombre a los eventos y a las operaciones de un sistema

Los eventos de un sistema (y sus operaciones asociadas) deberían expresarse en el nivel de propósito y no el medio físico de entrada o en el nivel de elementos de la interfaz.

También mejora la claridad si el nombre de un evento del sistema comienza con un verbo (agregar..., introducir..., terminar..., efectuar...), como en la figura 13.7, porque recalca que los eventos están orientados a comandos.

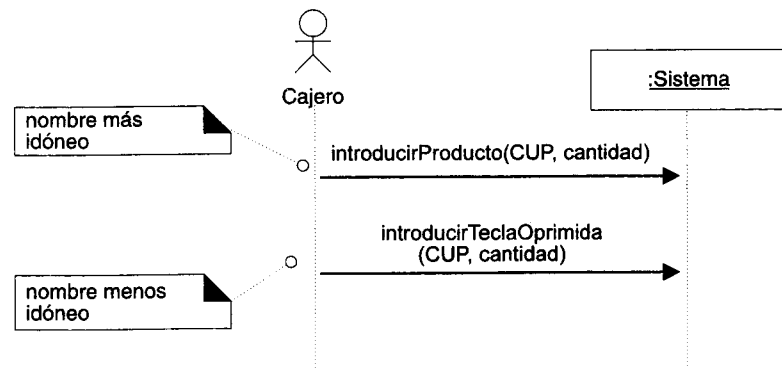


Figura 13.7 Escoja los nombres de los eventos y de las operaciones en un nivel abstracto.

Así, “terminarVenta” es preferible a “introducirTeclaOprimida” porque capta mejor el propósito de la operación: mantiene un carácter abstracto y no se pronuncia respecto a las decisiones de diseño sobre cuál interfaz sirve para capturar el evento del sistema.

En cuanto a expresar las operaciones en el nivel de propósito, procure alcanzar el nivel más alto o la meta final de asignar nombre a la operación. Por ejemplo, respecto a la operación que captura el pago:

<i>introducirImporteOfrecido(monto)</i>	deficiente
<i>introducirPago(monto)</i>	mejor
<i>introducirPago(monto)</i>	quizá mejor aún

13.11 Presentación del texto del caso de uso

En ocasiones conviene mostrar al menos fragmentos del texto del caso de uso dentro del diagrama de la secuencia, con el fin de describir gráficamente su estrecha relación con el caso de uso (figura 13.8).

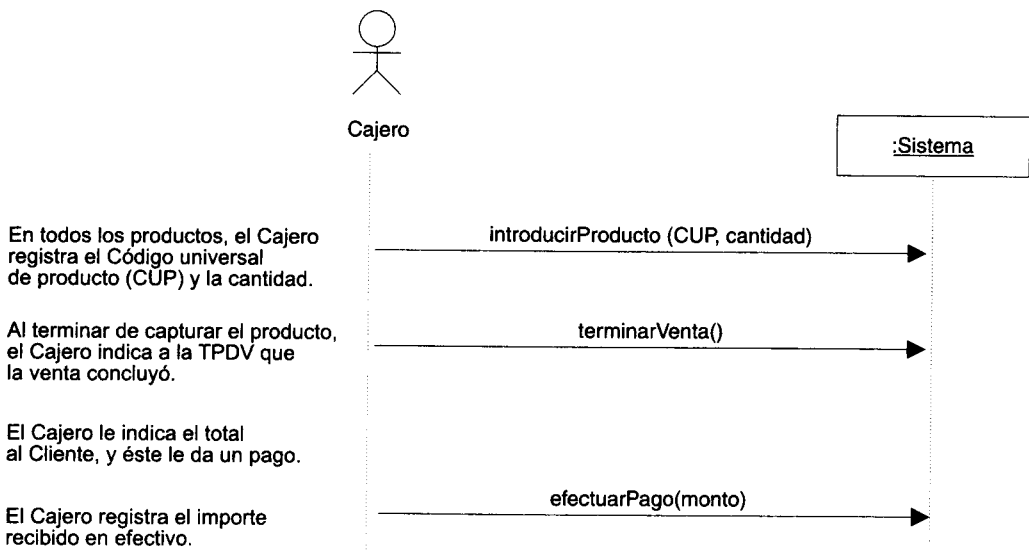


Figura 13.8 Diagrama de la secuencia de un sistema con texto del caso de uso.

13.12 Modelos muestra

Los diagramas de la secuencia de un sistema forman parte del modelo del comportamiento del sistema, como se advierte en la figura 13.9, la cual especifica a qué eventos responde un sistema y qué responsabilidades y poscondiciones tiene sus operaciones correspondientes.

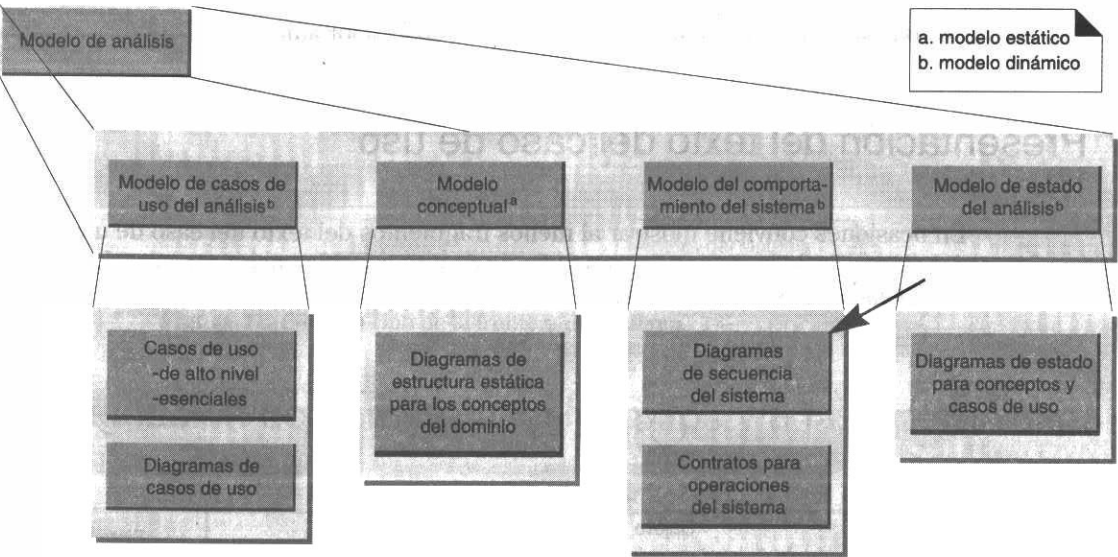


Figura 13.9 Modelo del análisis.

COMPORTAMIENTO DE LOS SISTEMAS: CONTRATOS

Objetivos

- Crear contratos para las operaciones de un sistema.

14.1 Introducción

Los contratos contribuyen a definir el comportamiento de un sistema; describen el efecto que sobre él tienen las operaciones. En este capítulo estudiaremos su utilización.

El lenguaje UML ofrece un soporte para definir los contratos, ya que permite definir las precondiciones y las poscondiciones de las operaciones.¹

14.2 Actividades y dependencias

Los contratos de operación del sistema se elaboran durante la fase de análisis en un ciclo de desarrollo. Su preparación depende del desarrollo previo del modelo conceptual, de los diagramas de la secuencia del sistema y la identificación de sus operaciones.

¹ En la definición formal del UML —o metamodelo—, las operaciones tienen un conjunto de propiedades predefinidas que incluye sus precondiciones y poscondiciones.

14.3 Comportamiento de un sistema

Antes de emprender el diseño lógico de cómo funcionará una aplicación de software, es necesario investigar y definir su comportamiento como “caja negra”. El **comportamiento de un sistema** es una descripción de *lo que* hace, sin explicar cómo lo hace. Los contratos son documentos muy útiles que describen el comportamiento de un sistema a partir de cómo cambia el estado de un sistema cuando se llama una operación suya.

14.4 Contratos

En la figura 14.1, el diagrama de la secuencia de un sistema muestra los eventos generados por un actor externo, pero no profundiza en los detalles de la funcionalidad asociada con las operaciones invocadas. No contiene los detalles necesarios para entender la respuesta del sistema, o sea su comportamiento.

En términos generales, un **contrato**¹ es un documento que describe lo que una operación se propone lograr. Suele redactarse en un estilo declarativo, enfatizando *lo que* sucederá y no *cómo* se conseguirá. Los contratos suelen expresarse a partir de los cambios de estado de las precondiciones y de las poscondiciones. Puede elaborarse un contrato para un método de una clase de software o para una operación más global del sistema.

El **contrato de operación del sistema** describe los cambios del estado del sistema total cuando se llama una de sus operaciones.



Figura 14.1 Las operaciones del sistema requieren las descripciones del contrato.

Repetimos: un contrato puede emplearse con una operación de alto nivel que se aplica al sistema entero o con un método de bajo nivel de una clase particular. Por ahora nos centraremos en su uso en las operaciones del sistema.

14.5 Ejemplo de contrato: introducirProducto

En el siguiente ejemplo se describe un contrato de la operación *introducirProducto* del sistema.

¹ Bertrand Meyer fue el primero en difundir el término.

Contrato	
Nombre:	introducirProducto (cup: número, cantidad: entero).
Responsabi- lidades:	Capturar (registrar) la venta de un producto y agregarla a la venta. Desplegar la descripción y el precio del producto.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R1.1, R1.3, R1.9. Casos de uso: Comprar productos.
Notas:	Utilizar el acceso superrápido a la base de datos.
Excepciones:	Si el CUP no es válido, indicar que se cometió un error.
Salida:	
Precondiciones:	El sistema conoce el CUP.
Poscondiciones:	
■ Si se trata de una nueva venta, se crea una <i>Venta</i> (creación de instancia). ■ Si se trata de una nueva venta, la nueva <i>Venta</i> fue asociada a <i>TPDV</i> (asociación formada). ■ Se creó una instancia <i>VentasLineadeProducto</i> (creación de instancia). ■ Se asoció una instancia <i>VentasLineadeProducto</i> a la <i>Venta</i> (asociación formada). ■ Se asignó cantidad a <i>VentasLineadeProducto.cantidad</i> (modificación de atributo). ■ Se asoció una instancia <i>VentasLineadeProducto</i> a la instancia <i>EspecificaciondeProducto</i>, basado esto en la correspondencia del CUP (asociación formada).	

14.6 Secciones del contrato

Una descripción de las secciones de un contrato se da en el siguiente esquema. No todas las secciones son necesarias; nosotros recomendamos las de *Responsabilidades* y *Poscondiciones*.

Contrato	
Nombre:	Nombre de la operación y parámetros.
Responsabi- lidades:	Descripción informal de las responsabilidades que debe cumplir la operación.
Tipo:	Nombre del tipo (concepto, clase de software, interfaz).

Referencias cruzadas:	Números de referencia de las funciones del sistema, casos de uso, etcétera.
Notas:	Notas de diseño, algoritmos e información afin.
Excepciones:	Casos excepcionales.
Salida:	No salidas de la Interfaz del Usuario; por ejemplo, men- sajes o registros que se envían afuera del sistema.
Precondiciones:	Suposiciones acerca del estado del sistema antes de ejecu- tar la operación.
Poscondiciones:	

- El estado del sistema después de la operación. Esto se explica a fondo en la siguiente sección.

14.7 Cómo preparar un contrato

Aplique la siguiente sugerencia para elaborar contratos.

- Para preparar un contrato en los casos de uso:
1. Identifique las operaciones del sistema a partir de los diagramas de su secuencia.
 2. Elabore un contrato en cada operación del sistema.
 3. Comience redactando la sección de *Responsabilidades*; después describa informalmente el propósito de la operación.
 4. Complete luego la sección de *Poscondiciones*, describiendo en forma declarativa los cambios de estado de los objetos en el modelo conceptual.
 5. Para describir las poscondiciones utilice las siguientes categorías:
 - ❑ Creación y eliminación de las instancias.
 - ❑ Modificación de los atributos.
 - ❑ Asociaciones formadas y canceladas.

14.7.1 Contratos y otros artefactos

- Los casos de uso sugieren los diagramas de los eventos y de la secuencia del sistema.
- Después se identifican las operaciones del sistema.

- El efecto de las operaciones del sistema se describe en los contratos.

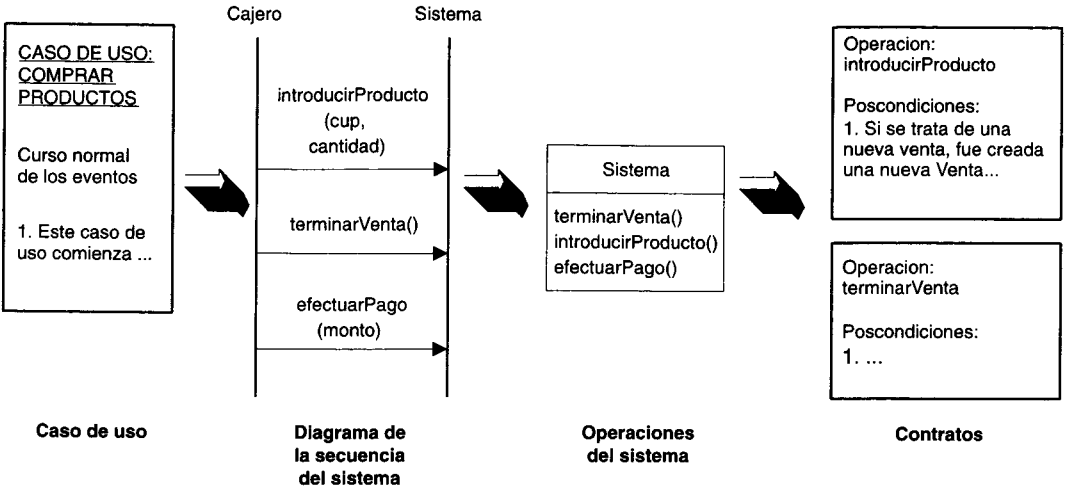


Figura 14.2 Contratos y otros artefactos.

14.8 Poscondiciones

Nótese que las poscondiciones en el ejemplo *introducirProducto* incluyen una clasificación; por ejemplo, *creación de instancias* o *asociación formada*. Después de la sección de *Responsabilidades*, la parte más importante del contrato son las poscondiciones, que estipulan cómo cambió el sistema tras esta operación. No son acciones que deben efectuarse durante la operación; más bien son declaraciones sobre el estado del sistema que se aplican una vez concluida la operación, es decir, *una vez que el humo se ha disipado*.

El UML no impone límites a la manera de expresar las poscondiciones; pero las siguientes categorías de cambio de estado han resultado de gran utilidad en la práctica. Le aconsejamos expresar sus poscondiciones de una manera similar.

Categorías útiles referentes a las poscondiciones del contrato:

- Creación y eliminación de las instancias.
- Modificación de los atributos.
- Asociaciones formadas y canceladas.

El UML no define cómo expresar las poscondiciones; así que puede usted escoger el formato que más le agrade. Lo importante es que adopte una actitud declarativa, orientada al cambio de estado y no a la acción, porque las poscondiciones deberían ser declaraciones sobre los estados o resultados, no una descripción de acciones a realizar.

14.8.1 Las poscondiciones se relacionan con el modelo conceptual

Las poscondiciones se expresan dentro del contexto del modelo conceptual. ¿Qué instancias es posible crear? La respuesta es: las provenientes de ese modelo. ¿Qué asociaciones es posible formar? La respuesta es: las que están en el modelo conceptual. Y así podríamos ir formulando y contestado más preguntas.

Cuando se elaboran contratos, generalmente el lector se percatará de la necesidad de registrar en el modelo conceptual nuevos conceptos, atributos o asociaciones. No se limite usted a la definición precedente del modelo conceptual: mejórelo conforme vaya haciendo más descubrimientos, mientras reflexiona sobre los contratos de las operaciones.

14.8.2 La ventaja de las poscondiciones

Expresado en una forma declarativa de cambios de estado, el contrato constituye una excelente herramienta de investigación, pues permite describir los cambios necesarios para que el sistema funcione sin necesidad de describir *cómo* se logran. En otras palabras, podemos posponer el diseño y la solución del software y concentrarnos analíticamente en *lo que* debe suceder, no en la manera de conseguirlo.

14.9 El espíritu de las poscondiciones: el escenario y el telón

Las poscondiciones deberían describir el estado de un sistema, no las acciones a realizar. Expréselas en tiempo pasado para enfatizar que se trata de declaraciones sobre un cambio pretérito de estado. Por ejemplo:

- Se **creó** una instancia *VentasLineadeProducto* (mejor);
- en lugar de
- Crear una instancia *VentasLineadeProducto* (peor).

Reflexione sobre las poscondiciones sirviéndose de la siguiente imagen:

El sistema y sus objetos se presentan en el escenario de un teatro.

1. Tome una fotografía del escenario antes de la operación.
2. Corra el telón del escenario y aplique la operación del sistema (*ruido de fondo con sonidos metálicos, gritos, chillidos...*).
3. Corra el telón y tome una segunda fotografía.

4. Compare las fotografías de antes y después, y exprese como poscondiciones los cambios del estado del escenario (*Se creó la instancia VentasLineadeProducto...*).

14.10 Explicación: poscondiciones de *introducirProducto*

En la siguiente sección analizaremos por qué se utilizan las poscondiciones de la operación del sistema de *introducirProducto*.

14.10.1 Creación y eliminación de instancias

Una vez que el cajero capturó el código universal de producto (CUP) y la cantidad de un producto, ¿qué nuevos objetos han de crearse? Si se trata de una nueva venta, habría que crear una instancia para una nueva *Venta*. Una instancia *VentasLineadeProducto* debería ser creada de modo incondicional. Por tanto:

- Si se trata de una nueva venta, se creó una *Venta* (creación de instancia).
- Se creó una instancia *VentasLineadeProducto* (creación de instancia).

14.10.2 Modificación de atributos

Una vez que la cajera capturó el código universal de producto y la cantidad de un producto, ¿qué atributos de los objetos nuevos o actuales deberían ser modificados? Habría que establecer la cantidad de *VentasLineadeProducto*. Por tanto:

- Se definió para *VentasLineadeProducto.cantidad* el valor de *cantidad* (modificación del atributo).

14.10.3 Asociaciones formadas y canceladas

Una vez que el cajero capturó el código universal de producto y la cantidad de un producto, ¿qué asociaciones entre los objetos nuevos y los actuales debieron haber sido formadas o canceladas? Habría que haber relacionado la nueva instancia *VentasLineadeProducto* con sus *Ventas* y con su *Producto*. Si se trataba de una nueva venta, la *Venta* debió haber sido relacionada con la *TPDV* dentro de la cual es registrada. Por tanto:

- Si se trata de una venta nueva, la nueva *Venta* fue asociada a la *TPDV* (asociación formada).
- Una instancia *VentasLineadeProducto* fue asociada a la *Venta* (asociación formada).
- Una instancia *VentasLineadeProducto* fue asociada a una *EspecificaciondeProducto*, con base en la correspondencia del CUP (asociación formada).

14.11 ¿Cuán completas deben ser las poscondiciones?

En la fase de análisis no es probable —ni siquiera necesario— generar un grupo de poscondiciones completas y exactas de la operación del sistema. Aconsejamos ver en su elaboración la conjetura inicial más acertada, a sabiendas de que los contratos estarán incompletos. Esta creación temprana —aunque incompleta— sin duda es preferible a posponer la investigación hasta la fase de diseño, cuando los creadores se concentrarán en el diseño de una solución más que en averiguar *lo que debe hacerse*.

Algunos de los detalles finos —y quizá hasta los más generales— se descubrirán durante la fase de diseño. No es algo necesariamente malo; en la fase de investigación se debilitan el esfuerzo y la dedicación si se prolongan demasiado tiempo. En la fase de diseño se logran algunos descubrimientos que después pueden guiar la fase de investigación de un ciclo iterativo posterior. Una de las ventajas del desarrollo iterativo es ésta: los descubrimientos hechos en la fase de diseño de un ciclo pueden mejorar la calidad de la investigación y el trabajo de análisis en el siguiente ciclo.

14.12 Descripción de los detalles y algoritmos del diseño: notas

La sección del contrato correspondiente a Notas es el lugar donde pueden hacerse las declaraciones del diseño referentes a la operación. Por ejemplo, si se sabe que se prefiere un algoritmo en particular para manejar la operación, esa sección es el sitio idóneo para su documentación.

14.13 Precondiciones

Las precondiciones definen las suposiciones sobre el estado del sistema al iniciarse la operación. Hay muchas precondiciones que pueden declararse en una operación, pero la experiencia revela que vale la pena mencionar las siguientes:

- Cosas que son importantes probar en el software en algún momento de la ejecución de la operación.
- Cosas que no serán sometidas a prueba, pero de las cuales depende el éxito de la operación. Queremos compartir esta suposición con los futuros lectores del contrato, a fin de subrayar su importancia y de que los lectores se percaten de ella.

14.14 Recomendación sobre cómo redactar contratos

- Una vez anotado el nombre de la operación, llene primero la sección de *Responsabilidades* y luego la de *Poscondiciones*, dejando al final la de *Precondiciones*. Son las tres secciones más importantes del proyecto en cuanto a su uso ulterior. Desde luego, si a un desarrollador no le resulta útil llenar una sección, no lo hará y nada pasará.
- Use la sección de *Notas* para explicar los detalles del diseño; por ejemplo, los algoritmos y los pasos secuenciales de alto nivel.
- Use la sección de *Excepciones* para explicar la reacción ante situaciones raras o especiales.
- Use las siguientes categorías de los cambios de estado en las poscondiciones:
 - Creación y eliminación de instancias.
 - Modificación de atributos.
 - Asociaciones formadas y canceladas.
- Exprese las poscondiciones en forma pasiva declarativa, en pretérito (*fue* registrado...) para destacar la declaración de un cambio de estado en vez del diseño de cómo iba a obtenerse. Por ejemplo:

Fue creada una instancia *VentasLineadeProducto* (bien).

es mejor que

Se creó una instancia *VentasLineadeProducto* (mal).

- La diferencia entre ambas oraciones puede parecer meramente académica y superficial; pero si se emplea la construcción activa en vez de la pasiva, sabemos por experiencia que los desarrolladores rápidamente adoptan la actitud de diseñar la forma en que van a resolver la operación del software. El espíritu del contrato es poner de relieve una declaración de los cambios de estado, absteniéndose de ofrecer los medios de una solución.
- No olvide establecer una memoria entre los objetos actuales y los de creación reciente, definiendo para ello la formación de una asociación. Por ejemplo, si no es suficiente que una nueva instancia *VentasLineadeProducto* se genere cuando ocurre la operación *IntroducirProducto*. Una vez finalizada la operación, deberá ser verdad que la instancia recién creada fue asociada a *Venta*; por tanto,
 - La instancia *VentasLineadeProducto* fue asociada a la *Venta* (asociación formada).

14.14.1 El error más frecuente en la preparación de contratos

El problema más común consiste en olvidar incluir la *formación de asociaciones*. Sobre todo cuando se crean nuevas instancias, muy probablemente será necesario haber establecido las asociaciones a varios objetos. No lo olvide.

14.15 Contratos para el caso de uso *Comprar productos*

14.15.1 Contrato para introducirProducto

Contrato	
Nombre:	introducirProducto (cup: número, cantidad: entero).
Responsabilidades:	Introducir (registrar) la venta de un producto y agregarlo a la venta. Desplegar la descripción del producto y su precio.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R1.1, R1.3, R1.9. Casos de uso: Comprar productos.
Notas:	Utilice el acceso superrápido a la base de datos.
Excepciones:	Si el CUP no es válido, indique que se cometió un error.
Salida:	
Precondiciones:	El sistema conoce el CUP.
Poscondiciones:	

- Si se trata de una nueva venta, una *Venta* fue creada (*creación de instancia*).
- Si se trata de una nueva venta, la nueva *Venta* fue asociada a la *TPDV* (*asociación formada o formación de asociaciones*).
- Se creó una instancia *VentasLineadeProducto* (*creación de instancia*).
- Se asoció *VentasLineadeProducto* a la *Venta* (*asociación formada*).
- Se estableció *VentasLineadeProducto.cantidad* con el valor de *cantidad* (*modificación de atributo*).
- La instancia *VentasLineadeProducto* fue asociada a una *EspecificaciondeProducto*, basado esto en la correspondencia del código universal de producto (*asociación formada*).

14.15.2 Contrato para terminarVenta

Contrato	
Nombre:	terminarVenta().
Responsabilidades:	Registrar que es el final de la captura de los productos de la venta y desplegar el total de la venta.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R1.2. Casos de uso: Comprar productos.
Notas:	
Excepciones:	Si no está realizándose una venta, indicar que se cometió un error.
Salida:	
Precondiciones:	El sistema conoce el CUP.
Poscondiciones:	

- Estableció *Venta.estaTerminada* en *verdadero* (modificación de atributo).

14.15.3 Contrato para efectuarPago

Contrato	
Nombre:	efectuarPago (monto: Número o Cantidad).
Responsabilidades:	Registrar el pago, calcular el saldo e imprimir el recibo.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R2.1. Casos de uso: Comprar productos.
Notas:	
Excepciones:	Si la venta no está concluida, indicar que se cometió un error. Si el monto es menor que la venta total, indicar que se cometió un error.
Salida:	
Precondiciones:	
Poscondiciones:	

- Se creó un *Pago* (creación de instancia).
- Se asignó a *Pago.montoOfrecido* el valor de *monto* (modificación de atributo).
- Se asoció el *Pago* a la *Venta* (relación formada).
- Se asoció la *Venta* a la *Tienda* para agregarla al registro histórico de las ventas terminadas (relación formada).

14.16 Contratos para el caso de uso Inicio

14.16.1 Contrato para Inicio

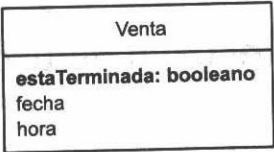
Contrato	
Nombre:	inicio().
Responsabilidades:	Inicializar el sistema.
Tipo:	Sistema.
Referencias cruzadas:	
Notas:	
Excepciones:	
Salida:	
Precondiciones:	
Poscondiciones:	

- Se creó una instancia *Tienda*, *TPDV*, *CatalogodeProductos* y *EspecificacionesdeProducto* (creación de instancia).
- Se asoció *CatalogodeProductos* a *EspecificacionesdeProducto* (asociación formada).
- Se asoció *Tienda* a *CatalogodeProductos* (asociación formada).
- Se asoció *Tienda* a *TPDV* (asociación formada).
- Se asoció *TPDV* a *CatalogodeProductos* (asociación formada).

14.17 Cambios del modelo conceptual

Estos contratos sugieren la existencia de un dato que todavía no ha figurado en el modelo conceptual: la terminación de la captura del producto en la venta. Lo modifica la especificación *terminarVenta*, y la especificación *efectuarPago* lo prueba como pre-condición.

Una forma de representar esta información es expresarla como un atributo *estaTerminada* (o *capturaEstaTerminada*) de la venta, por medio de un valor booleano:



Se cuenta con otras soluciones alternas para representar el estado cambiante del sistema. Un método es el **patrón de estado**, que estudiaremos en un capítulo subsecuente.

14.18 Modelos muestra

Los contratos que rigen las operaciones del sistema forman parte del modelo del comportamiento del sistema, el cual describe la interfaz externa y el comportamiento de todo el sistema (figura 14.3).

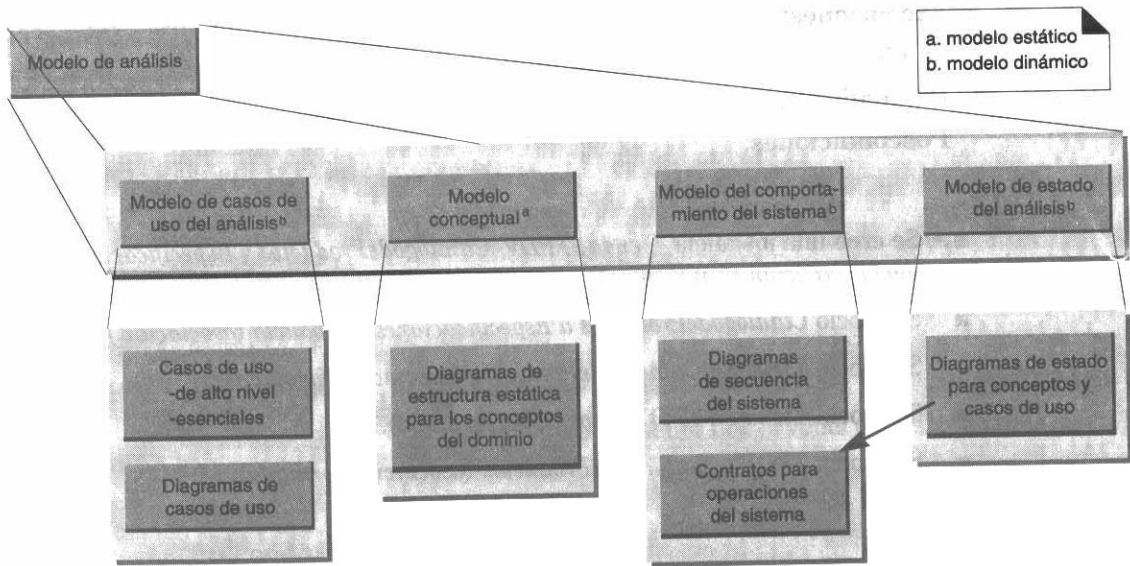


Figura 14.3 Modelo de análisis.

PARTE IV FASE DEL DISEÑO (1)

DEL ANÁLISIS AL DISEÑO

Objetivos

- Motivar la transición del análisis al diseño.

15.1 Conclusión de la fase de análisis

En la fase de análisis del desarrollo se da prioridad al conocimiento de los requerimientos, los conceptos y las operaciones relacionadas con el sistema. A menudo la investigación y el análisis se caracterizan por centrarse en cuestiones concernientes al *qué*: cuáles son los procesos, los conceptos, etcétera.

En el UML hay otros artefactos que sirven para capturar los resultados de una investigación; a continuación se describe un grupo mínimo de ellos (que ya estudiamos en capítulos anteriores):

Artefacto de análisis	Preguntas que se contestan
Casos de uso	¿Cuáles son los procesos del dominio?
Modelo conceptual	¿Cuáles son los conceptos, los términos?
Diagramas de la secuencia de un sistema	¿Cuáles son los eventos y las operaciones del sistema?
Contratos	¿Qué hacen las operaciones del sistema?

15.2 Inicio de la fase de diseño

Durante el ciclo de desarrollo iterativo es posible pasar a la fase de diseño, una vez terminados estos documentos del análisis. Durante este paso se logra una solución lógica que se funda en el paradigma orientado a objetos. Su esencia es la elaboración de **diagramas de interacción**, que muestran gráficamente cómo los objetos se comunicarán entre ellos a fin de cumplir con los requerimientos.

El advenimiento de los diagramas de interacción nos permite dibujar **diagramas de diseño de clases** que resumen la definición de las clases (e interfaces) implementables en software.

En los siguientes capítulos examinaremos la creación de estos artefactos. Los diagramas de interacción son los más importantes de ellos (desde el punto de vista de la preparación de un buen diseño) y exigen gran dedicación y esfuerzo creativos. Para prepararlos hay que aplicar los principios de la asignación de **responsabilidades** y utilizar los **patrones de diseño**. Por tanto, en los siguientes capítulos nos centraremos en esos principios y patrones del diseño orientado a objetos.

La creación de los diagramas de interacción exige conocer:

- Los principios de la asignación de responsabilidades.
- Los patrones del diseño.

DESCRIPCIÓN DE LOS CASOS REALES DE USO

Objetivos

- Crear casos reales de uso.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECAS
MONTEVIDEO - URUGUAY

16.1 Introducción

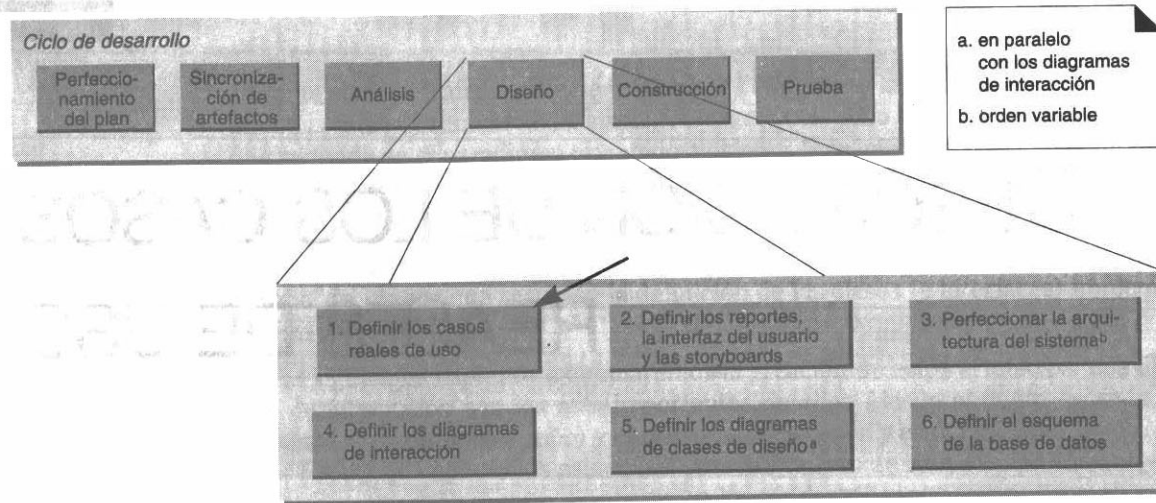
Los casos reales de uso presentan un diseño concreto de cómo se realizará el caso. En el presente capítulo vamos a examinar su creación.

16.2 Actividades y dependencias

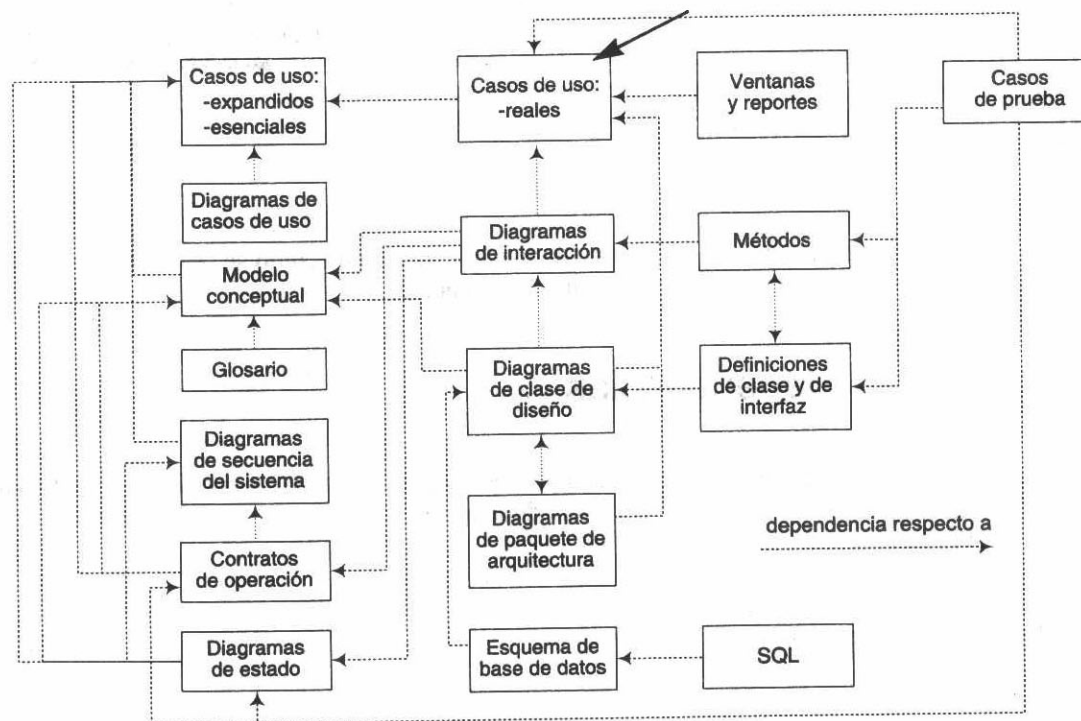
La definición de los casos de uso reales es una de las primeras actividades dentro de un ciclo de desarrollo. Su creación depende de los casos esenciales conexos que hayan sido generados antes.

16.3 Casos reales de uso

Un **caso real de uso** describe el diseño concreto del caso de uso a partir de una tecnología particular de entrada y salida, así como de su implementación global. Por ejemplo, si interviene una interfaz gráfica para el usuario, el caso de uso real incluirá



Actividades de la fase de diseño dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

diagramas de las ventanas en cuestión y una explicación de la interacción de bajo nivel con los artefactos de la interfaz.

Tal vez no sea necesario generarlos. Una alternativa podría consistir en que el diseñador realizara storyboards o secuencias de las pantallas de la interfaz general para el usuario y que después fuera incorporando los detalles durante la implementación.

Los storyboards son de gran utilidad, si antes de la implementación los diseñadores o el cliente necesitan descripciones rigurosamente detalladas.

16.4 Ejemplo: Comprar productos: versión 1

En el siguiente ejemplo y en la figura 16.1, observe la utilización de un esquema de codificación con los artefactos de ventana para obtener una descripción fluida.

Casos de uso: **Comprar productos: versión 1**
 (efectivo exclusivamente)

Actores: Cliente (iniciador), Cajero.

Propósito: Capturar una venta y su pago en efectivo.

Resumen: Un Cliente llega a la caja con productos que desea comprar. El Cajero registra los productos de la compra y recibe el pago en efectivo. Al terminar la transacción, el Cliente se marcha con los productos comprados.

Tipo: Primario y real.

Referencias cruzadas: *Funciones:* R1.1, R1.2, R1.3, R1.7, R1.9, R2.1.

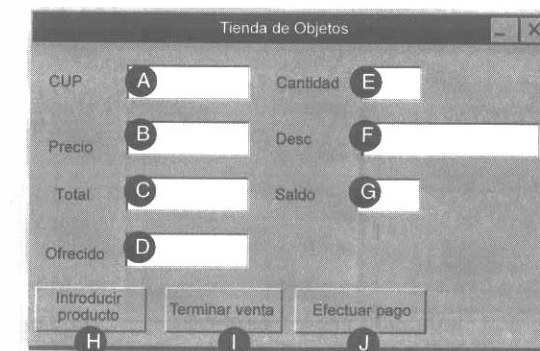


Figura 16.1 Ventana-1.

Curso normal de los eventos	
Acción de los actores	Respuesta del sistema
1. Este caso comienza cuando un Cliente llega a la caja TPDV con objetos que desea comprar.	
2. Con cada producto, el Cajero teclea el código universal de producto (CUP) en A de la Ventana-1. Si hay más de un producto, es opcional capturar la cantidad en E. Se oprime H después de capturar cada producto.	3. Agrega la información sobre el producto a la actual transacción de ventas. La descripción y el precio del producto actual se muestran en B y en F de la Ventana-1.
4. Al terminar de capturar los productos, el Cajero oprime el botón I para indicarle a la TPDV que terminó de capturar los productos.	5. Calcula y presenta en C el total de la venta. 6. ...

16.5 Modelos muestra

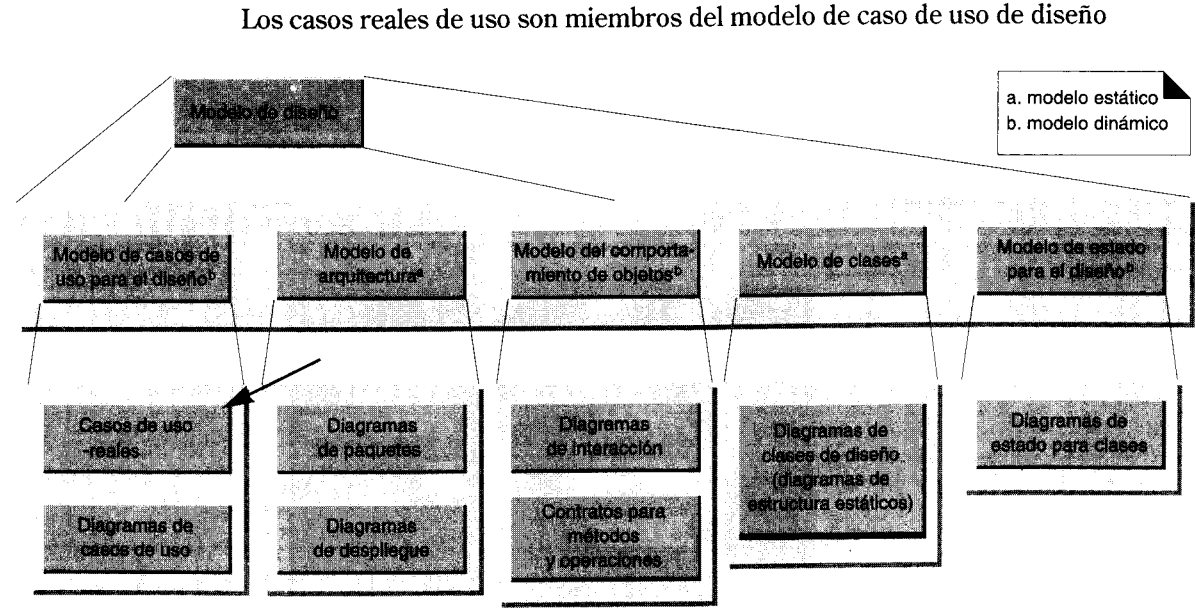


Figura 16.2 El modelo de diseño.

DIAGRAMAS DE COLABORACIÓN

Objetivos

- Leer la notación del UML para un diagrama de colaboración.

17.1 Introducción

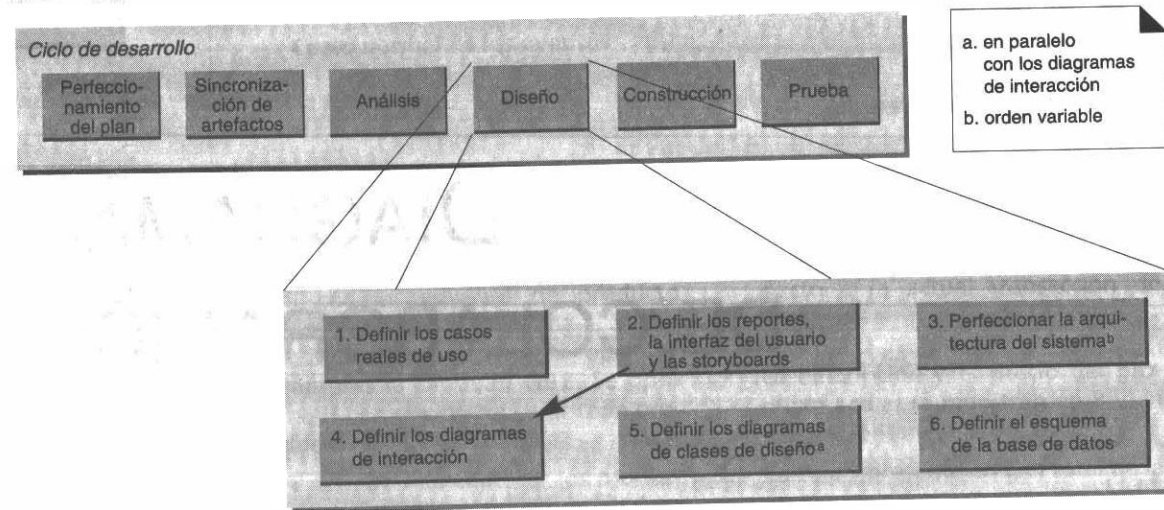
En los contratos de colaboración se incluye una primera conjetura óptima sobre las poscondiciones referentes al *inicio* de las operaciones del sistema: *inicio*, *introducirProducto*, *terminarVenta* y *efectuarPago*. Sin embargo, los contratos no muestran una solución de cómo los objetos de software van a cumplir con ellas.

El UML contiene **diagramas de interacción** que explican gráficamente cómo los objetos interactúan a través de mensajes para realizar las tareas. En el presente capítulo examinaremos su creación para la aplicación del punto de venta.

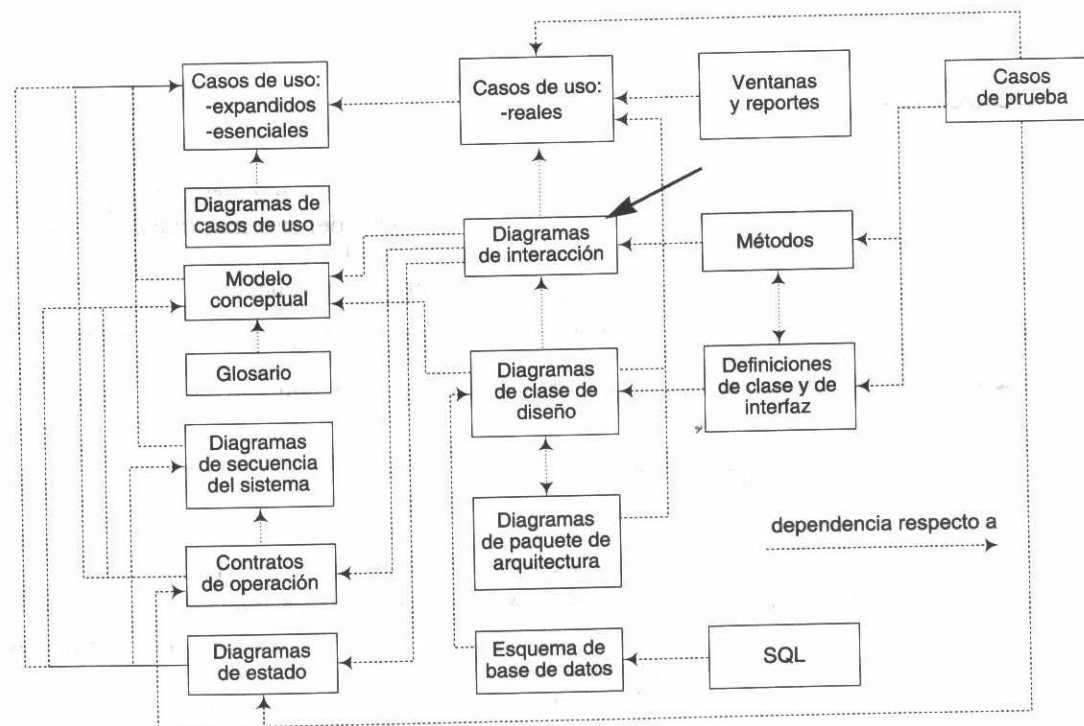
17.2 Actividades y dependencias

Los diagramas de interacción se realizan en la fase de diseño de un ciclo de desarrollo. No se pueden preparar si antes no se generan los siguientes artefactos:

- Un modelo conceptual: a partir de este modelo el diseñador podrá definir las clases del software correspondientes a los conceptos. Los objetos de las clases participan en las interacciones que se describen gráficamente en los diagramas.
- Contratos de la operación del sistema: a partir de ellos el diseñador identifica las responsabilidades y las poscondiciones que han de llenar los diagramas de interacción.



Actividades de la fase de diseño dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

- Casos de uso reales (o esenciales): a partir de ellos el diseñador recaba información sobre las tareas que realizan los diagramas de interacción, además de lo estipulado en los contratos.

17.3 Diagramas de interacción

Un **diagrama de interacción** explica gráficamente las interacciones existentes entre las instancias (y las clases) del modelo de éstas. El punto de partida de las interacciones es el cumplimiento de las poscondiciones de los contratos de operación.

El UML define dos tipos de estos diagramas; ambos sirven para expresar interacciones semejantes o idénticas de mensaje:

1. diagramas de colaboración
2. diagramas de secuencia

Los **diagramas de colaboración** describen las interacciones entre los objetos en un formato de grafo o red, como se aprecia en la figura 17.1.

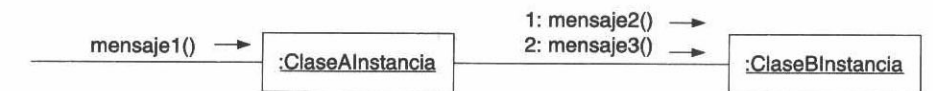


Figura 17.1 Diagrama de colaboración.

Los **diagramas de secuencia** describen las interacciones en una especie de formato de cerca o muro, como se observa en la figura 17.2.

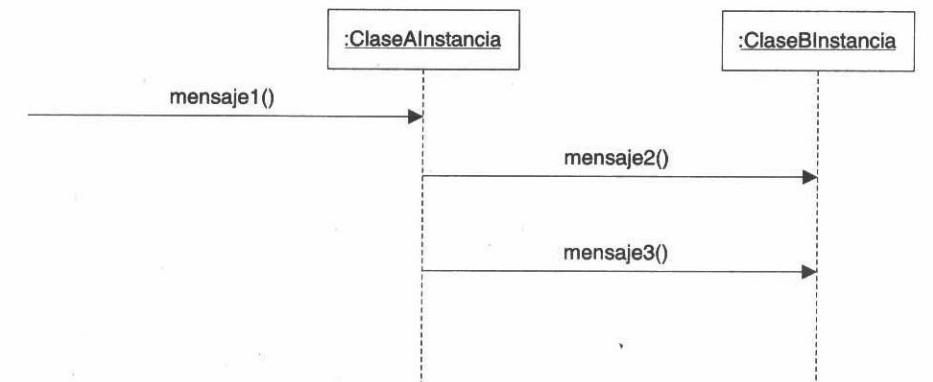


Figura 17.2 Diagrama de secuencia.

En este capítulo y a lo largo del libro nos centraremos en los diagramas de colaboración por su excepcional expresividad, su capacidad de comunicar más información

contextual y su economía de espacio.¹ Pero recuerde el lector que tanto una como otra notación pueden expresar conceptos parecidos.

17.4 Ejemplo de un diagrama de colaboración: efectuarPago

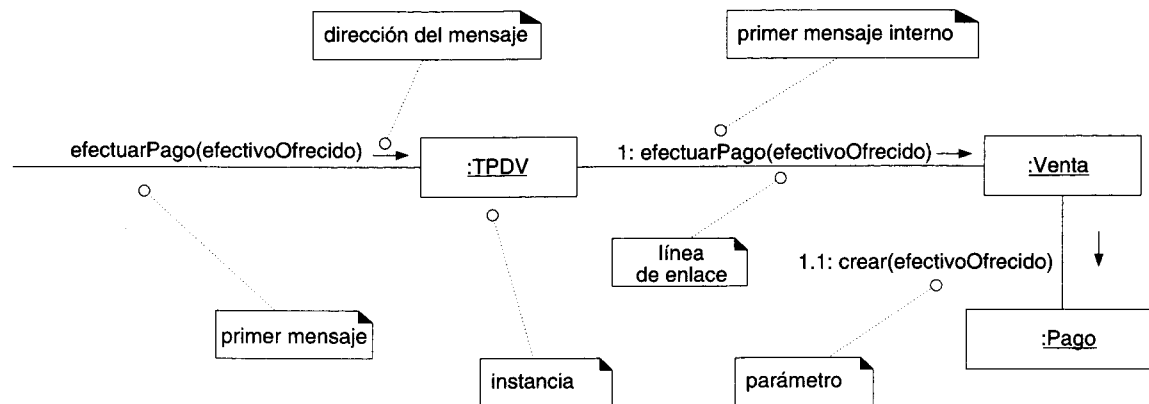


Figura 17.3 Diagrama de colaboración.

El diagrama de colaboración de la figura 17.3 se lee así:

1. El mensaje *efectuarPago* se envía a una instancia de *TPDV*. La instancia corresponde al mensaje *efectuarPago* de la operación del sistema.
2. El objeto *TPDV* envía el mensaje *efectuarPago* a la instancia *Venta*.
3. El objeto *Venta* crea una instancia de un *Pago*.

17.5 Los diagramas de interacción son un artefacto de gran utilidad

Un problema frecuente en los proyectos de la tecnología de objetos consiste en no comprender la utilidad de preparar los diagramas de interacción, la consideración cuidadosa de la asignación de responsabilidades y las habilidades que todo esto requiere. Son muy importantes la asignación de responsabilidades y el diseño de la colaboración entre los objetos.

Un porcentaje considerable de las actividades del proyecto ha de destinarse a esa fase. Más aún, es fundamentalmente durante ella cuando se exige aplicar las habilidades del diseño en los patrones, las expresiones y los principios.

¹ Los diagramas de colaboración nos permiten decir más en un espacio que los diagramas de secuencia y expresar además más información contextual; por ejemplo, el tipo de visibilidad entre los objetos. También resulta más fácil expresar la lógica condicional y la concurrencia.

En cierto modo, la creación de los casos de uso, de los modelos conceptuales y de otros artefactos resulta más fácil que asignar responsabilidades y elaborar diagramas bien diseñados de interacción. Ello se debe a lo siguiente: los sutiles principios del diseño en que se fundan son muchos más numerosos que cualquier análisis orientado a objetos o que cualquier artefacto de diseño.

Los diagramas de interacción constituyen uno de los artefactos más importantes que se generan en el análisis y en el diseño orientados a objetos.

El tiempo y el esfuerzo dedicados a su preparación deberían absorber un porcentaje considerable de la actividad total destinada al proyecto.

Para mejorar la calidad de su diseño, es posible aplicar patrones, principios y expresiones codificados.

Los principios del diseño necesarios para construir eficazmente los diagramas de interacción *pueden* codificarse, explicarse y aplicarse de modo metódico. Esta forma de entender y utilizar los principios del diseño se basa en **patrones**: directrices y principios estructurados. Por tanto, luego de exponer la sintaxis de los diagramas de interacción, nos ocuparemos (en capítulos posteriores) de los patrones de diseño y de su aplicación a los diagramas de interacción.

17.6 Éste es un capítulo dedicado exclusivamente a la notación

En este capítulo nos proponemos describir y resumir la notación del UML para un tipo particular de diagrama de interacción: el diagrama de colaboración. No explicaremos los principios ni las directrices que rigen la elaboración de un diagrama de colaboración bien diseñado.

No es indispensable que el lector conozca todos los detalles de la notación que se ofrecen en este capítulo para proseguir el estudio del libro. No obstante, le recomendamos echar un vistazo a los ejemplos y familiarizarse un poco con ellos antes de pasar a los siguientes capítulos.

17.7 Lea las directrices de diseño en los siguientes capítulos

Hay varios principios de diseño que deben conocerse para poder crear buenos diagramas de interacción. Tras familiarizarse un poco con la notación de éstos, conviene leer los siguientes capítulos dedicados a esos principios y a la forma de aplicarlos.

17.8 Cómo preparar diagramas de colaboración

Aplique las siguientes normas cuando elabore un diagrama de colaboración.

Para preparar un diagrama de colaboración:

1. Elabore un diagrama por cada operación del sistema durante el ciclo actual de desarrollo.
 - En cada mensaje del sistema, dibuje un diagrama incluyéndolo como mensaje inicial.
2. Si el diagrama se torna complejo (por ejemplo, si no cabe holgadamente en una hoja de papel de 8.5 x 11), divídalo en diagramas más pequeños.
3. Diseñe un sistema de objetos interactivos que realicen las tareas, usando como punto de partida las responsabilidades del contrato de operación, las poscondiciones y la descripción de casos de uso. Aplique el GRASP y otros patrones para desarrollar un buen diseño.

17.8.1 Los diagramas de colaboración y otros artefactos

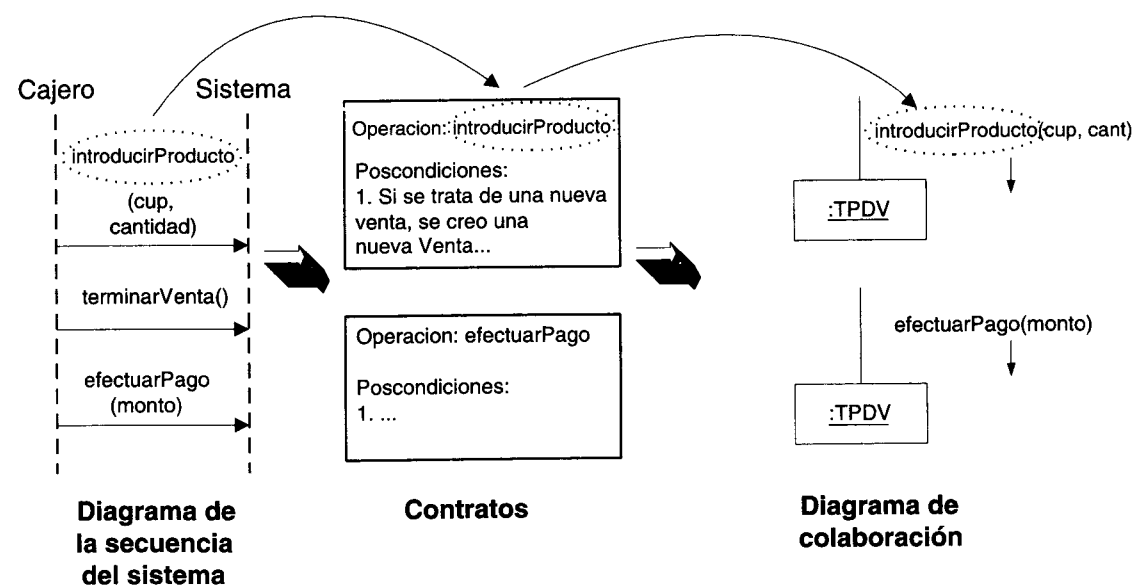


Figura 17.4 Relación entre los artefactos.

Como se observa en la figura 17.4, la relación entre los artefactos incluye lo siguiente:

- Los casos de uso indican los eventos del sistema que se muestran explícitamente en los diagramas de su secuencia.
- En los contratos se describe la mejor conjetura inicial sobre las operaciones del sistema.
- Las operaciones del sistema representan mensajes y éstos originan diagramas que explican gráficamente cómo los objetos interactúan para llevar a cabo las funciones requeridas.

17.9 Notación básica de los diagramas de colaboración

17.9.1 Representación gráfica de las clases y de las instancias

El UML ha adoptado un método simple y uniforme de describir visualmente las instancias para distinguirlas de los tipos (figura 17.5):

- Con cada tipo de elemento del UML (clase, actor, ...), una instancia utiliza el mismo símbolo gráfico usado para representar el tipo, pero se subraya el texto.

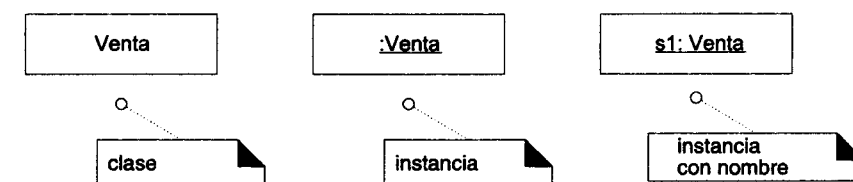


Figura 17.5 Clase e instancias.

Por tanto, para incluir la instancia de una clase en un diagrama de interacción, se recurre al símbolo gráfico usual de la casilla de la clase, sólo que el nombre se subraya. Además, en un diagrama de colaboración, al nombre de la clase siempre se le anteponen dos puntos.

Finalmente, un nombre de instancia sirve para identificarla de modo inequívoco.

17.9.2 Representación gráfica de los vínculos

El **vínculo** (o enlace) es una trayectoria de conexión entre dos instancias; indica alguna forma de navegación y visibilidad que es posible entre las instancias (figura 17.6). En un lenguaje más formal, el vínculo es una instancia de una asociación. Si vemos dos instancias en una relación de cliente/servidor, una trayectoria de navegación del cliente al servidor significa que los mensajes pueden enviarse del primero al segundo. Así, existe un vínculo —o trayectoria de navegación— entre *TPDV* y una *Venta*, a lo largo del cual pueden fluir los mensajes; por ejemplo, el mensaje *agregarPago*.

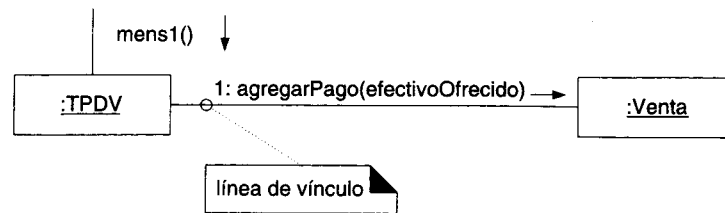


Figura 17.6 Líneas de vínculo (enlace).

17.9.3 Representación gráfica de los mensajes

Los mensajes entre objetos pueden representarse por medio de una flecha con un nombre y situada sobre una línea del vínculo. A través de éste puede fluir un número indefinido de mensajes (figura 17.7). Se agrega un número de secuencia que indique el orden consecutivo de los mensajes en la serie actual de control.

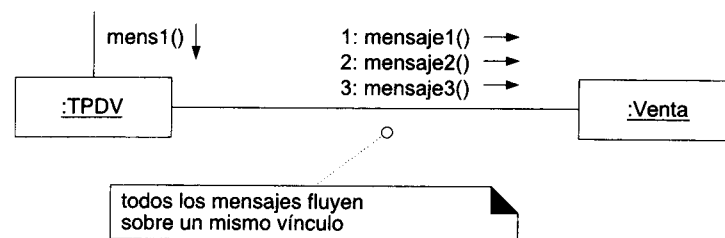


Figura 17.7 Mensajes.

17.9.4 Representación gráfica de los parámetros

Los parámetros de un mensaje pueden anotarse dentro de paréntesis después del nombre del mensaje (figura 17.8). Es opcional incluir o no el tipo de parámetro en cuestión.

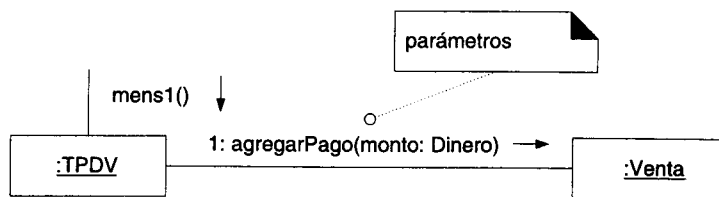


Figura 17.8 Parámetros.

17.9.5 Representación gráfica del mensaje de devolver valor

Puede incluirse un valor de retorno anteponiéndole al mensaje un nombre de variable de esa instrucción y un operador de asignación (':=') (figura 17.9). Es opcional mostrar el tipo del valor de retorno.

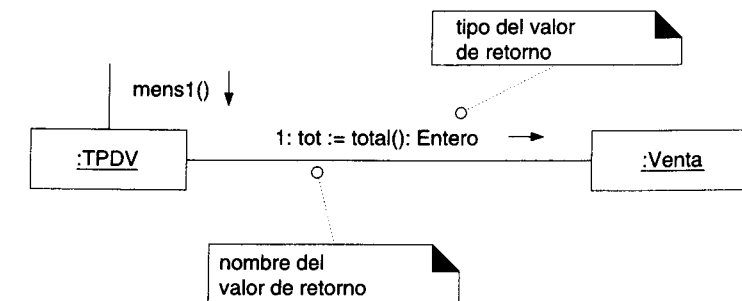


Figura 17.9 Devolver valores.

17.9.6 Sintaxis de los mensajes

El lenguaje UML cuenta con una sintaxis estándar para los mensajes:

```
retorno ::= mensaje(parametro : tipoParametro) : tipoRetorno
```

No obstante, como se aprecia en la figura 17.10, es legal servirse de otra sintaxis como la de Java o la de Smalltalk. Recomendamos emplear la sintaxis estándar de UML a fin de que los diagramas de colaboración sigan siendo un lenguaje relativamente independiente.

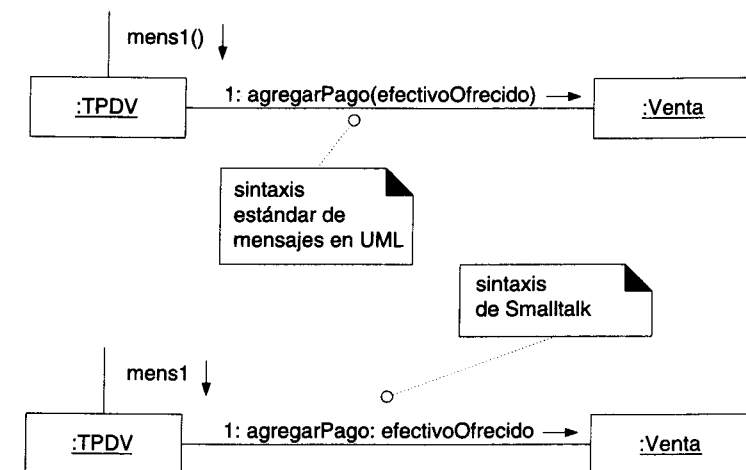


Figura 17.10 Los mensajes pueden expresarse en varias sintaxis.

17.9.7 Representación gráfica de los mensajes al "emisor" o a "esto"

Puede enviarse un mensaje de un objeto a sí mismo (figura 17.11).

Esto lo muestra gráficamente un vínculo consigo mismo, donde el mensaje fluye a lo largo del vínculo.

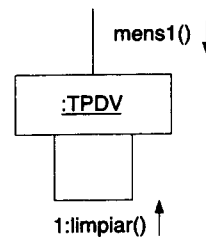


Figura 17.11 Mensajes a "esto".

17.9.8 Representación gráfica de la iteración

La iteración se indica posponiendo un asterisco (*) al número de secuencia.

Ese símbolo significa que, dentro de un ciclo, el mensaje va a ser enviado repetidamente al receptor (figura 17.12).

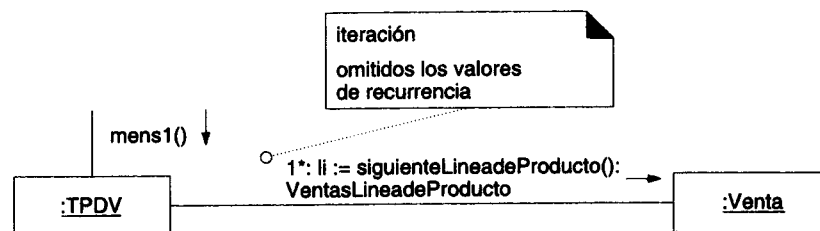


Figura 17.12 Iteración.

También es posible incluir una cláusula de iteración que indique los valores de recurrencia (figura 17.13).

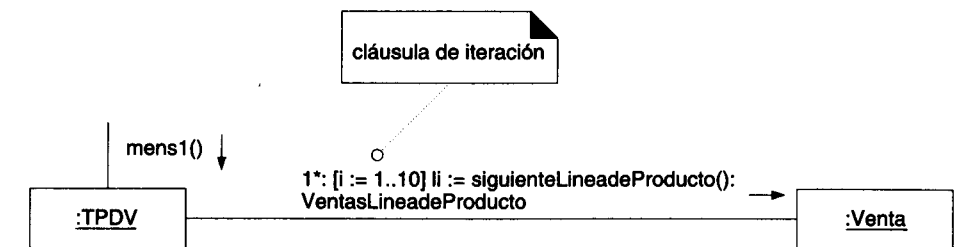


Figura 17.13 Cláusula de iteración.

Si se expresa más de un mensaje que ocurre dentro de la misma cláusula de iteración (por ejemplo, una serie de mensajes en un ciclo *for*), se repetirá la cláusula con cada mensaje (figura 17.14).

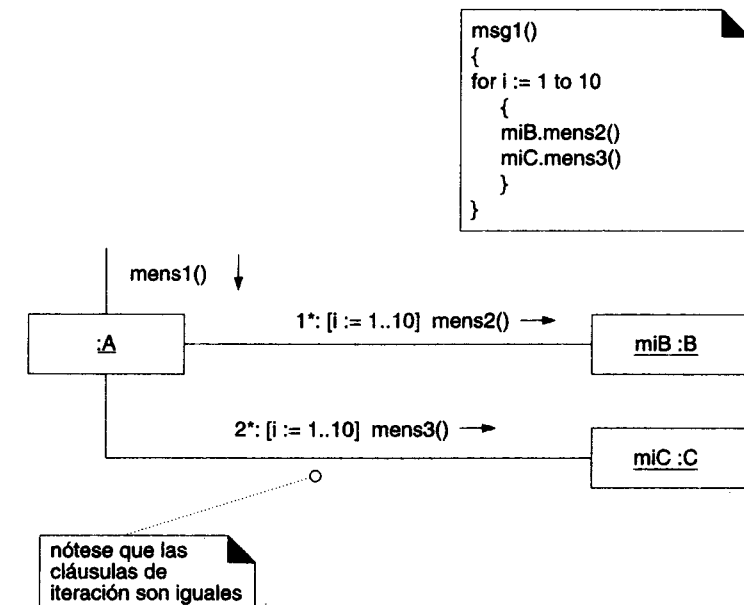


Figura 17.14 Mensajes múltiples dentro de la misma cláusula de iteración.

17.9.9 Representación gráfica de la creación de instancias

El mensaje de creación independiente del lenguaje es *crear*, que se muestra en el momento de ser enviado a la instancia que vamos a generar (figura 17.15).

Aunque en la mayoría de los lenguajes orientados a objetos, las instancias suelen generarse utilizando un mensaje *nuevo* (u operador) con la clase y no con una instancia, esta notación ocupa menos espacio aunque no sea completamente exacta.

Es opcional que la nueva instancia contenga o no un símbolo «nuevo».¹
El mensaje *crear* puede contener parámetros, lo cual indica la transferencia de los valores iniciales. En Java, de ese modo se indican —por ejemplo— los parámetros de constructores.

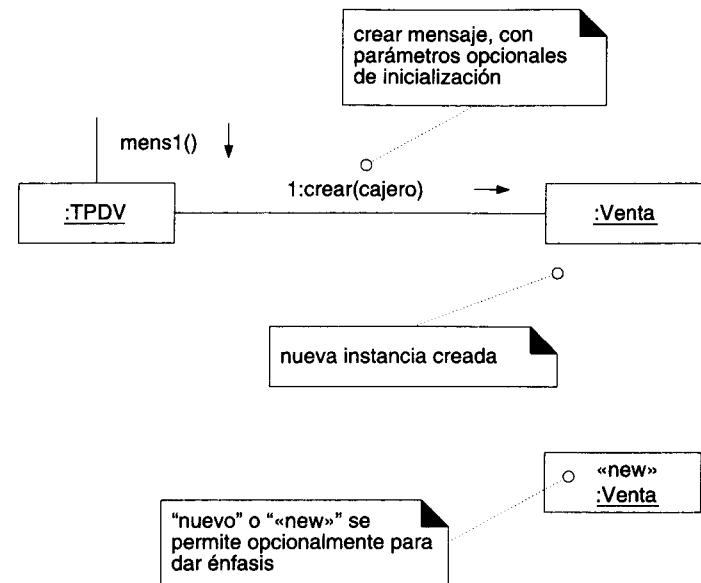


Figura 17.15 Creación de instancias.

En varios lenguajes, el mensaje *crear* se traduce así:

Lenguaje	Significado de crear ()
C++	Asignación automática u operador <i>new</i> seguido de una llamada a un constructor.
Java	operador <i>new</i> seguido de una llamada a un constructor.
Smalltalk	mensaje <i>new</i> o una variación de <i>new</i> : seguido del mensaje <i>initialize</i> .

¹ Un **estereotipo** del UML que estudiaremos más adelante.

17.9.10 Representación gráfica de la secuencia de número de los mensajes

El orden de los mensajes se indica con un **número de secuencia**, como se aprecia en la figura 17.16. El esquema de la numeración es:

- 1. El primer mensaje no se numera. Así, *mens1()* no lleva número.
- 2. El orden y el anidamiento de los mensajes siguientes se indican con un esquema legal de numeración, donde a los mensajes anidados se les ha antepuesto un número. La anidación se denota anteponiendo el número del mensaje de entrada al del mensaje de salida.

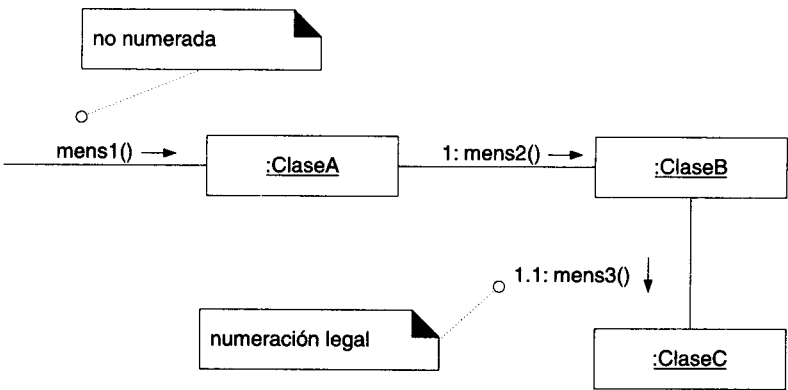


Figura 17.16 Numeración de secuencias.

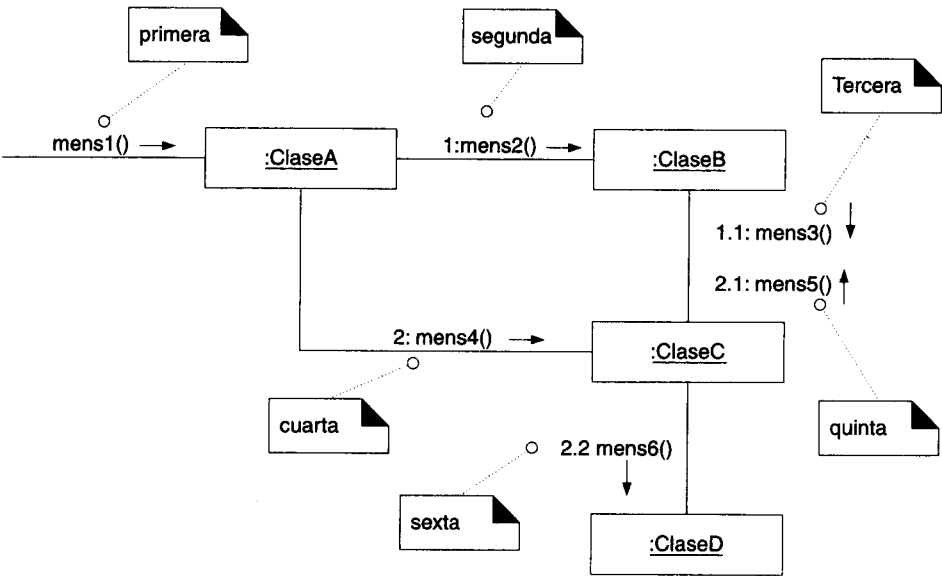


Figura 17.17 Numeración compleja de secuencias.

17.9.11 Representación gráfica de los mensajes condicionales

Un mensaje condicional (figura 17.18) se indica posponiendo al número de la secuencia una cláusula condicional entre corchetes, en forma parecida a como se hace con una cláusula de iteración. El mensaje se envía sólo si la cláusula se evalúa como *verdadera*.

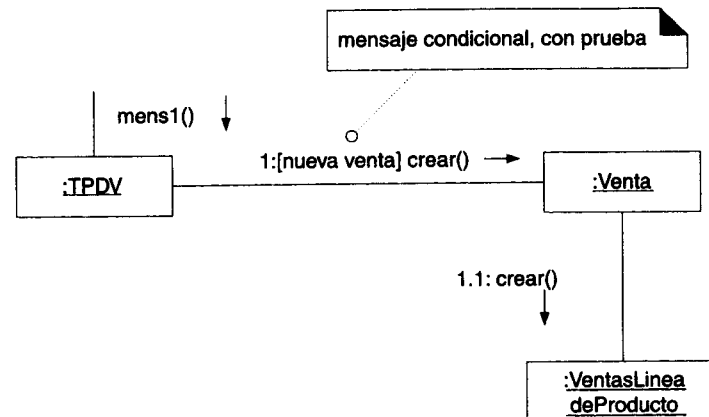


Figura 17.18 Mensaje condicional.

17.9.12 Representación gráfica de trayectorias condicionales mutuamente excluyentes

El ejemplo de la figura 17.19 contiene los números de secuencia con trayectorias condicionales que se excluyen mutuamente.

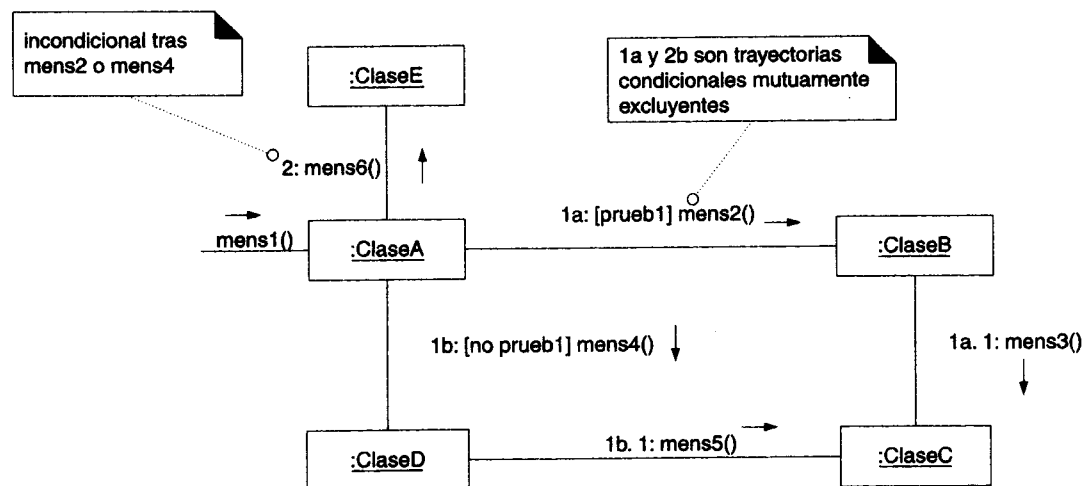


Figura 17.19 Mensajes mutuamente excluyentes.

En este caso es necesario modificar las expresiones de la secuencia con una letra de la trayectoria condicional. Por convención, la primera letra en usarse es *a*. La figura 17.19 establece qué tanto *1a* como *1b* podrían ejecutarse después de *mens1()*. Ambas son el número de la secuencia 1 porque pueden ser el primer mensaje interno.

Nótese que a los subsiguientes mensajes anidados todavía se les sigue anteponiendo la secuencia de sus mensajes externos. Así *1b.1* es un mensaje anidado dentro de *1b*.

17.9.13 Representación gráfica de las colecciones

Un **multiobjeto**, o conjunto de instancias, puede dibujarse como un icono de pila según se observa en la figura 17.20.

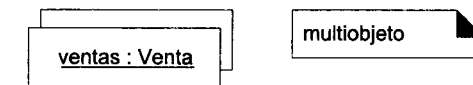


Figura 17.20 Un multiobjeto.

Un multiobjeto suele implementarse como un grupo de instancias guardadas en un contenedor u objeto colección; por ejemplo, un *vector* de la STL de C++, un *Vector* de Java o una *ColeccionOrdenada* (*OrderedCollection*) de Smalltalk. Pero no necesariamente se implementa así; representa tan sólo un conjunto lógico de instancias.

17.9.14 Representación gráfica de los mensajes dirigidos a multiobjetos

Un mensaje dirigido a un icono de multiobjeto indica que se envía al objeto colección. Así, en la figura 17.21 el mensaje *tamaño* está siendo enviado a la instancia *java.util.Vector* para solicitar el número de elementos del *Vector*.

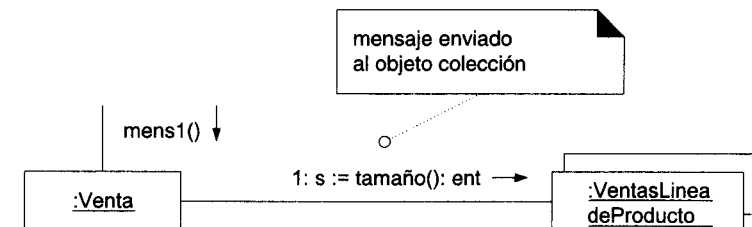


Figura 17.21 Mensaje dirigido a un multiobjeto.

En el lenguaje UML, los mensajes dirigidos a un multiobjeto *no* se transmite a todos los elementos (como sucedía en las versiones anteriores del UML).

En la figura 17.22 se explican gráficamente los mensajes a un multiobjeto y a un elemento.

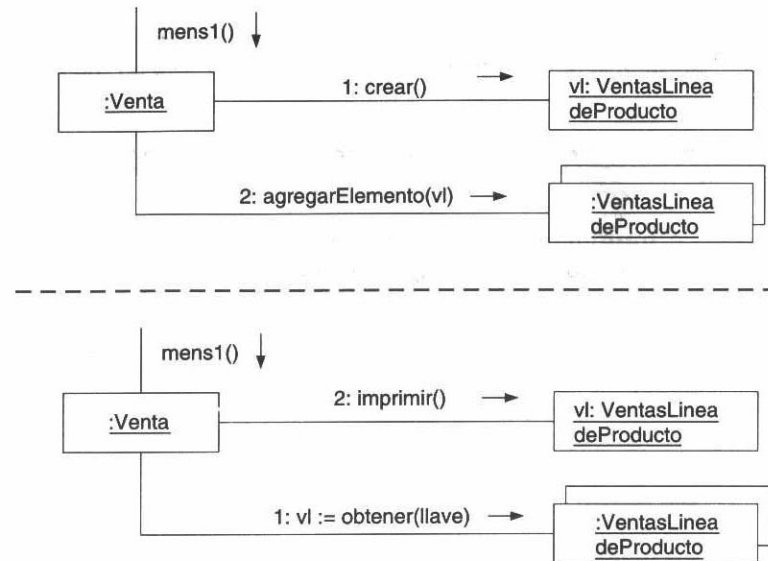


Figura 17.22 Mensajes dirigidos a un multiobjeto y a un elemento.

17.9.15 Representación gráfica de los mensajes dirigidos a un objeto clase

Los mensajes pueden ser dirigidos a la propia clase y no a una instancia, con el fin de llamar los métodos de la clase. Por ejemplo, en Java éstos se implementan como métodos estáticos; en Smalltalk son métodos de clases.

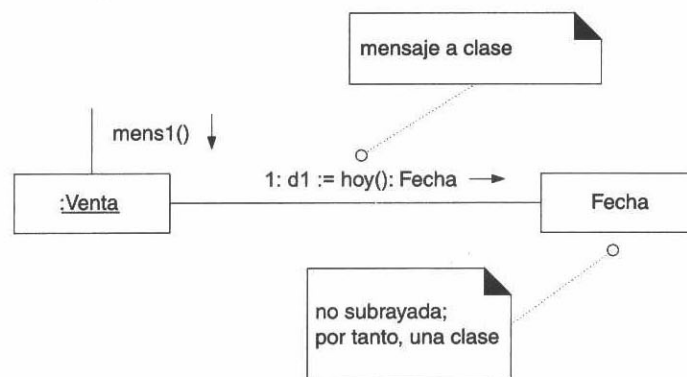


Figura 17.23 Mensajes a un objeto clase (llamada a método estático).

Los mensajes se representan de un modo muy simple: se incluyen dentro de una casilla de clases cuyo nombre no esté subrayado; se indica con ello que el mensaje va a ser enviado a una clase y no a una instancia (figura 17.23).

En consecuencia, es importante ser consistente en subrayar los nombres de las instancias cuando se desea denotar una instancia; pues de lo contrario pueden interpretarse erróneamente los mensajes dirigidos a las instancias y los dirigidos a las clases.

17.10 Modelos muestra

Los diagramas de interacción son miembros del modelo del comportamiento de objetos, ya que describen el comportamiento de los objetos del software.

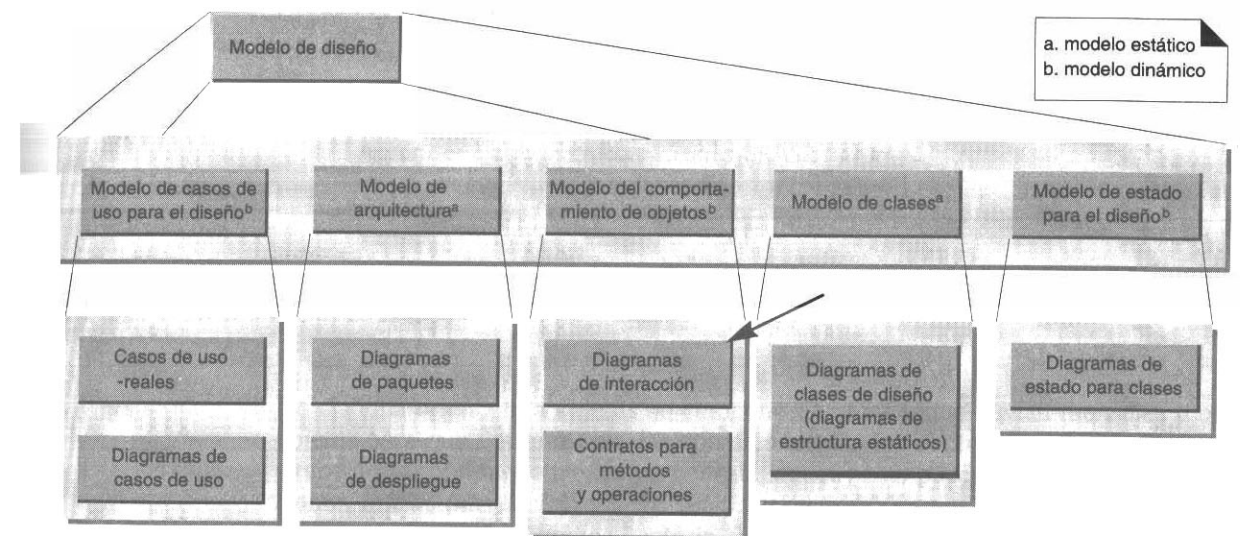


Figura 17.24 El modelo de diseño.

GRASP: PATRONES PARA ASIGNAR RESPONSABILIDADES

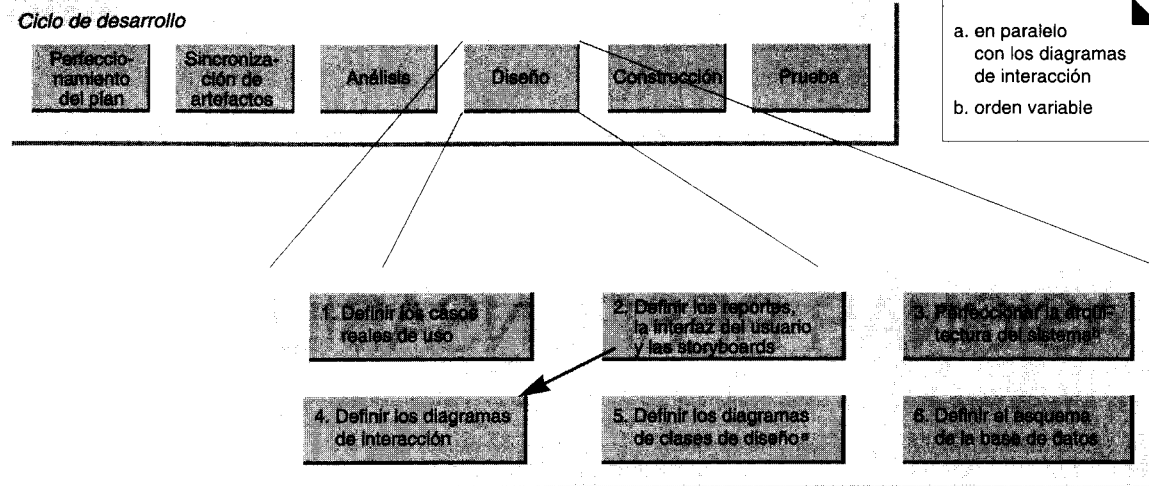
Objetivos

- Definir patrones.
- Aprender a aplicar cinco patrones GRASP.

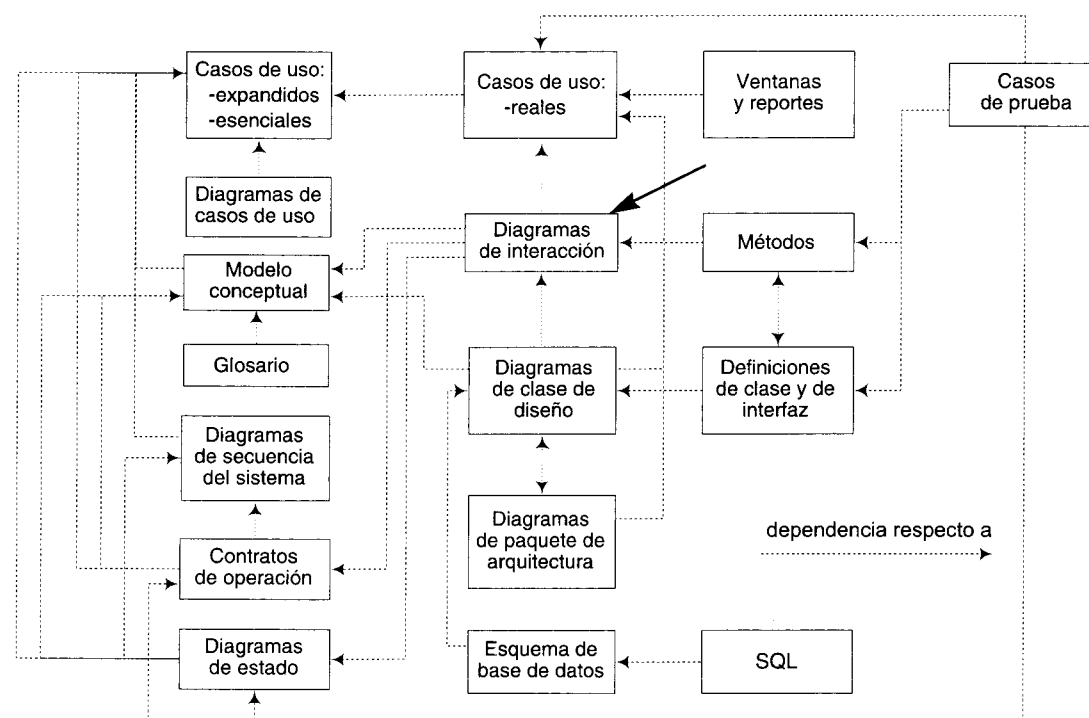
18.1 Introducción

Un sistema orientado a objetos se compone de objetos que envían mensajes a otros objetos para que lleven a cabo las operaciones. En los contratos se incluye una conjetura inicial óptima sobre las responsabilidades y las poscondiciones de las operaciones *inicio*, *introducirProducto*, *terminarVenta* y *efectuarPago*. Los diagramas de interacción describen gráficamente la solución —a partir de los objetos en interacción— que estas responsabilidades y poscondiciones satisfacen.

La calidad de diseño de la interacción de los objetos y la asignación de responsabilidades presentan gran variación. Las decisiones poco acertadas dan origen a sistemas y componentes frágiles y difíciles de mantener, entender, reutilizar o extender. Una implementación hábil se funda en los principios cardinales que rigen un buen diseño orientado a objetos. En los patrones GRASP se codifican algunos de ellos, que se aplican al preparar los diagramas de interacción, cuando se asignan las responsabilidades o durante ambas actividades.



Actividades de la fase de diseño dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

18.2 Actividades y dependencias

Los patrones a los que nos referimos se aplican durante la elaboración de los diagramas de interacción, al asignar las responsabilidades a los objetos y al diseñar la colaboración entre ellos.

18.3 Los diagramas de interacción bien diseñados son muy útiles

A continuación repetimos algunos puntos ya expuestos en el capítulo anterior cuando hablamos de los diagramas de colaboración:

- Los diagramas de interacción son algunos de los artefactos más importantes que se preparan en el análisis y diseño orientados a objetos.
- Es muy importante asignar acertadamente las responsabilidades al momento de elaborar los diagramas de interacción.
- El tiempo y el esfuerzo que se dedican a su elaboración, así como un examen riguroso de la asignación de responsabilidades, deberían absorber parte considerable de la fase de diseño de un proyecto.
- Los patrones, principios y expresiones especializadas codificados sirven para mejorar la calidad del diseño.

Los principios del diseño que se requieren para construir buenos diagramas de interacción pueden codificarse, explicarse y utilizarse en forma metódica. Esta manera de entender y usar los principios del diseño se funda en los *patrones con que se asignan las responsabilidades*.

18.4 Responsabilidades y métodos

Booch y Rumbaugh definen la **responsabilidad** como “un contrato u obligación de un tipo o clase” [BJR97]. Las responsabilidades se relacionan con las obligaciones de un objeto respecto a su comportamiento. Esas responsabilidades pertenecen, esencialmente, a las dos categorías siguientes:

1. conocer
2. hacer

Entre las responsabilidades de un objeto relacionadas con **hacer** se encuentran:

- hacer algo en uno mismo
- iniciar una acción en otros objetos

- controlar y coordinar actividades en otros objetos

Entre las responsabilidades de un objeto relacionadas con **conocer** se encuentran:

- estar enterado de los datos privados encapsulados
- estar enterado de la existencia de objetos conexos
- estar enterado de cosas que se puede derivar o calcular

Las responsabilidades se asignan a los objetos durante el diseño orientado a objetos. Por ejemplo, puede declararse que “una *Venta* es responsable de imprimirse ella misma” (un hacer) o que “una *Venta* tiene la obligación de conocer su fecha” (un conocer). Las responsabilidades relacionadas con “conocer” a menudo pueden inferirse del modelo conceptual por los atributos y asociaciones explicadas en él.

La granularidad de la responsabilidad influye en su traducción a clases y métodos. La responsabilidad de “brindar acceso a las bases relacionales de datos” puede incluir docenas de clases y cientos de métodos. En cambio, la de “imprimir una venta” tal vez no incluya más que un método o unos cuantos.

Responsabilidad no es lo mismo que método: los métodos se ponen en práctica para cumplir con las responsabilidades. Éstas se implementan usando métodos que operen solos o en colaboración con otros métodos y objetos. Así, la clase *Venta* podría definir uno o varios métodos que imprimen una instancia *Venta*; digamos el método *imprimir*. Para cumplir con esa responsabilidad, la *Venta* puede colaborar con otros objetos, entre ellos el envío de un mensaje a los objetos *VentasLineadeProducto*, pidiéndoles que se impriman ellos mismos.

18.5 Las responsabilidades y los diagramas de interacción

Este capítulo tiene por objeto ayudarle al lector a aplicar los principios fundamentales que rigen la asignación de responsabilidades a objetos. En los artefactos del UML, las responsabilidades (implementadas como métodos) suelen tenerse en cuenta al momento de preparar los diagramas de interacción, cuya notación ya examinamos en el capítulo anterior.

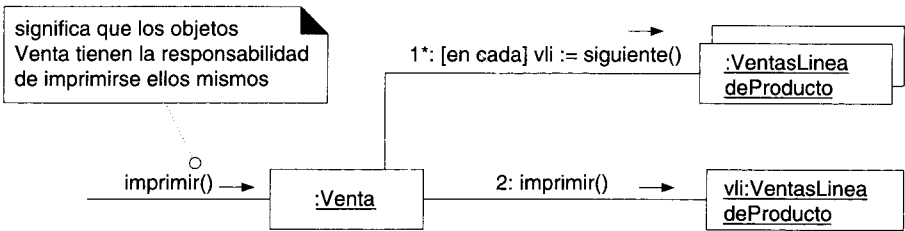


Figura 18.1 Las responsabilidades y los métodos están relacionados.

La figura 18.1 indica que a los objetos *Venta* se les ha asignado la responsabilidad de imprimirse ellos mismos, la cual se llama con un mensaje *imprimir* y se cumple con el método correspondiente *imprimir*. Más aún, para atender esta responsabilidad hay que colaborar con los objetos *VentasLineadeProducto*, pidiéndoles que impriman.

En resumen, los diagramas de interacción muestran las decisiones referentes a la asignación de responsabilidades entre los objetos. Cuando se preparan, se toman decisiones sobre la asignación que se reflejan en los mensajes que son enviados a varias clases de objetos. En el presente capítulo expondremos ampliamente los principios fundamentales —expresados en los patrones GRASP— para guiar las decisiones sobre la asignación de responsabilidades. Esas decisiones se reflejarán después en los diagramas de interacción.

18.6 Patrones

Los diseñadores expertos en orientación a objetos (y también otros diseñadores de software) van formando un amplio repertorio de principios generales y de expresiones que los guían al crear software. A unos y a otras podemos asignarles el nombre de **patrones**, si se codifican en un formato estructurado que describe el problema y su solución, y si se les asigna un nombre. A continuación ofrecemos un ejemplo de patrón.

Nombre del patrón:	Experto.
Solución:	Asignar una responsabilidad a la clase que tiene la información necesaria para cumplirla.
Problema que resuelve:	¿Cuál es el principio fundamental en virtud del cual asignaremos las responsabilidades a los objetos?

En la terminología de objetos, el **patrón** es una descripción de un problema y su solución que recibe un nombre y que puede emplearse en otros contextos; en teoría, indica la manera de utilizarlo en circunstancias diversas.¹ Muchos patrones ofrecen orientación sobre cómo asignar las responsabilidades a los objetos ante determinada categoría de problemas.

Expresado lo anterior con palabras más simples, el **patrón** es una pareja de problema/solución con un nombre y que es aplicable a otros contextos, con una sugerencia sobre la manera de usarlo en situaciones nuevas.

“El patrón de un individuo puede ser la estructura primitiva de otra persona”, máxima con que en la tecnología de objetos se explica la vaguedad de los patrones [GHJV94]. En nuestro estudio de los patrones no abordaremos el tema de lo que conviene llamar patrón; nos centraremos en el valor pragmático de utilizar ese estilo como medio de presentar y recordar los principios tan útiles de la ingeniería del software.

¹ La notación formal de los patrones nació con los patrones arquitectónicos de Christopher Alexander [AIS77]. En los años ochenta, Kent Beck y Ward Cunningham hicieron su aplicación al software [Beck94, Coplien95].

18.6.1 Los patrones no suelen contener ideas nuevas

Los patrones no se proponen descubrir ni expresar nuevos principios de la ingeniería del software. Todo lo contrario: intentan codificar el conocimiento, las expresiones y los principios *ya existentes*: cuanto más trillados y generalizados, tanto mejor. En consecuencia, los patrones GRASP —que describiremos aquí— no introducen ideas novedosas; son una mera codificación de los principios básicos más usados.

18.6.2 Los patrones tienen nombre

En teoría, todos los patrones poseen nombres muy sugestivos. El asignar nombre a un patrón, a un método o a un principio ofrece las siguientes ventajas:

- Apoya el agrupamiento y la incorporación del concepto a nuestro sistema cognitivo y a la memoria.
- Facilita la comunicación.

Darle nombre a una idea compleja —digamos a un patrón— es un ejemplo de la fuerza de la abstracción: convierte una forma compleja en una forma simple con sólo eliminar los detalles. Por tanto, los patrones GRASP poseen nombres concisos como *Experto*, *Creador*, *Controlador*.

18.6.3 La asignación de nombre a los patrones mejora la comunicación

Cuando se le da nombre a un patrón, un simple nombre nos permite discutir con otros un principio en todos sus aspectos. Considere la siguiente conversación entre dos diseñadores de software que emplean una nomenclatura común de patrones (*Experto*, *Separacion Vista-Modelo* y otros términos afines) para tomar una decisión sobre un diseño:

Alfredo: “En tu opinión, ¿a qué objeto deberíamos asignar la responsabilidad de imprimir una *Venta*? Creo que convendría asignarla a *Separacion Vista-Modelo*. ¿Qué te parece una *VistadeReportedeVentas*?”

Teresa: “En mi opinión, *Experto* es una mejor solución por ser una impresión simple y porque Ventas tiene toda la información que se requiere en salida impresa. Creo que le asignaremos esta responsabilidad a la *Venta*.”

Alfredo: “Está bien, hagámoslo.”

Agrupar las expresiones especializadas y los principios del diseño con nombres de uso común facilita la comunicación y le confiere a la búsqueda un nivel de abstracción más alto.

18.7 GRASP: patrones de los principios generales para asignar responsabilidades

Resumimos a continuación la introducción anterior:

- Asignar correctamente las responsabilidades es muy importante en el diseño orientado a objetos.
- La asignación de responsabilidades a menudo se asignan en el momento de preparar los diagramas de interacción.
- Los patrones son parejas de problema/solución con un nombre, que codifican buenos principios y sugerencias relacionados frecuentemente con la asignación de responsabilidades.

Dicho esto, podemos proceder a examinar los patrones GRASP.

Pregunta:	¿Qué son los patrones GRASP?
Respuesta:	Los patrones GRASP describen los principios fundamentales de la asignación de responsabilidades a objetos, expresados en forma de patrones.

Es importante entender y poder aplicar estos principios durante la preparación de un diagrama de interacción, pues un diseñador de software sin mucha experiencia en la tecnología de objetos debe dominarlos cuanto antes: constituyen el fundamento de cómo se diseñará el sistema.

GRASP es un acrónimo que significa **General Responsibility Assignment Software Patterns** (patrones generales de software para asignar responsabilidades).¹ El nombre se eligió para indicar la importancia de *captar* (*grasping*) estos principios, si se quiere diseñar eficazmente el software orientado a objetos.

18.7.1 Cómo aplicar los patrones GRASP

En las siguientes secciones explicaremos los primeros cinco patrones de GRASP:

- Experto
- Creador
- Alta Cohesión
- Bajo Acoplamiento
- Controlador

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

¹ Desde el punto de vista técnico, deberíamos escribir “Patrones GRAS” en vez de “Patrones GRASP” pero la segunda expresión suena mejor en inglés, idioma en que fue acuñado el término.

Hay otros patrones de los que nos ocuparemos en un capítulo posterior, pero vale la pena dominar los cinco anteriores porque se refieren a cuestiones y a aspectos fundamentales del diseño.

Por favor, estudie los patrones que siguen; observe cómo se emplean en los diagramas muestra de interacción y luego aplíquelos al preparar otros diagramas. Comience dominando *Experto*, *Creador*, *Controlador*, *Alta Cohesión* y *Bajo Acoplamiento*. Más tarde irá aprendiendo los patrones restantes.

18.8 La notación del UML para los diagramas de clase

Los diagramas de clase en UML presentan las clases de software en contraste con los conceptos de dominio. La casilla de clase consta de tres secciones; la tercera contiene los métodos de la clase, como se aprecia en la figura 18.2.

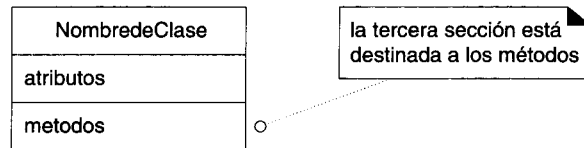


Figura 18.2 Las clases del software muestran los nombres de los métodos.

Los detalles de esta notación se abordan en un capítulo ulterior. En la siguiente exposición sobre los patrones, utilizaremos esporádicamente esta modalidad de la casilla de clase.

18.9 Experto

Solución **Asignar una responsabilidad al experto en información: la clase que cuenta con la información necesaria para cumplir la responsabilidad.**

Problema ¿Cuál es el principio fundamental en virtud del cual se asignan las responsabilidades en el diseño orientado a objetos?

Un modelo de clase puede definir docenas y hasta cientos de clases de software, y una aplicación tal vez requiera el cumplimiento de cientos o miles de responsabilidades. Durante el diseño orientado a objetos, cuando se definen las interacciones entre los objetos, tomamos decisiones sobre la asignación de responsabilidades a las clases. Si se hacen en forma adecuada, los sistemas tienden a ser más fáciles de entender, mantener y ampliar, y se nos presenta la oportunidad de reutilizar los componentes en futuras aplicaciones.

Ejemplo En la aplicación del punto de venta, alguna clase necesita conocer el gran total de la venta.

Comience asignando las responsabilidades con una definición clara de ellas.

A partir de esta recomendación se plantea la pregunta:

¿Quién es el responsable de conocer el gran total de la venta?

Desde el punto de vista del patrón Experto, deberíamos buscar la clase de objetos que posee la información necesaria para calcular el total. Examinemos detenidamente el modelo conceptual parcial de la figura 18.3.

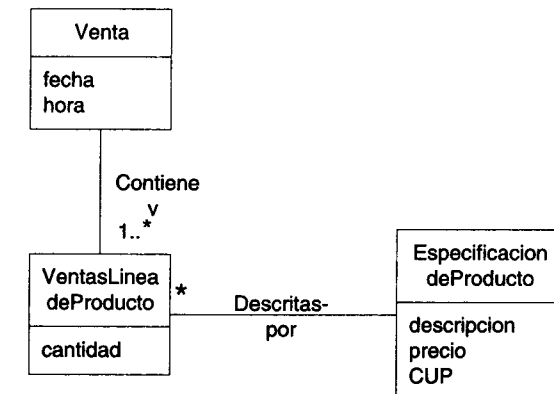


Figura 18.3 Asociaciones de la Venta.

¿Qué información hace falta para calcular el gran total? Hay que conocer todas las instancias *VentasLineadeProducto* de una venta y la suma de sus subtotales. Y esto lo

conoce únicamente la instancia *Venta*; por tanto, desde el punto de vista del Experto, *Venta* es la clase correcta de objeto para asumir esta responsabilidad; es el *experto en información*.

Como se señaló con anterioridad, es dentro del contexto de la preparación de los diagramas de interacción (por ejemplo, los de colaboración) donde surgen estas cuestiones concernientes a la responsabilidad. Imagine que vamos a empezar a dibujar diagramas para asignar responsabilidades a los objetos. El diagrama parcial de colaboración y el de clases de la figura 18.4 describen gráficamente las decisiones que hemos adoptado hasta ahora.

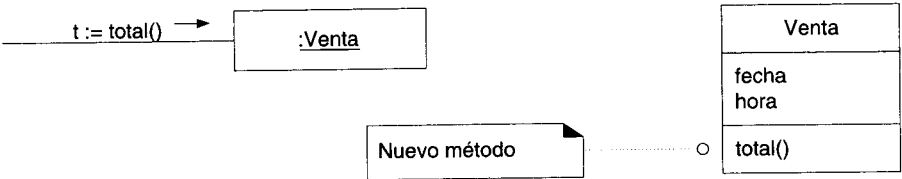


Figura 18.4 Diagrama parcial de colaboración.

Todavía no terminamos. ¿Qué información hace falta para determinar el subtotal de la línea de productos? Se necesitan *VentasLineadeProducto.cantidad* y *EspecificaciondeProducto.precio*. *VentasLineadeProducto* conoce su cantidad y su correspondiente *EspecificaciondeProducto*; por tanto, desde la perspectiva de patrón Experto, *VentasLineadeProducto* debería calcular el subtotal; es el *experto en información*.

Por lo que respecta a un diagrama de colaboración, lo anterior significa que la *Venta* necesita enviar mensajes de *subtotal* a cada *VentasLineadeProducto* y sumar los resultados; el diseño en cuestión se incluye en la figura 18.5.

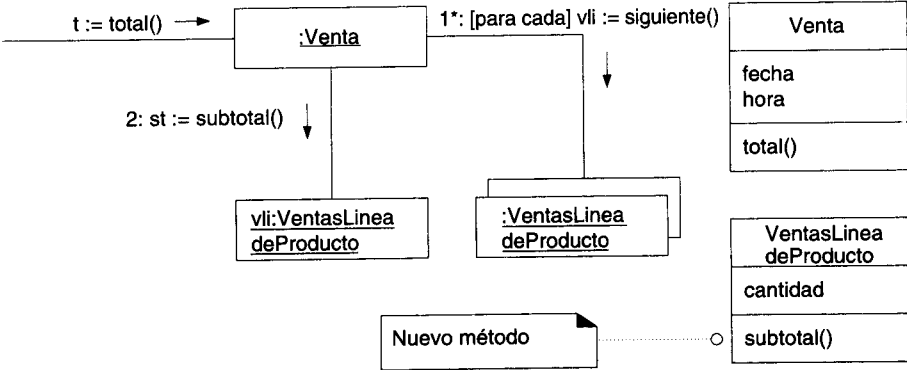


Figura 18.5 Cálculo del total de la venta.

VentasLineadeProducto no puede cumplir la responsabilidad de conocer y dar el subtotal, si no conoce el precio del producto. *EspecificaciondeProducto* es un Experto en información para contestar su precio; por tanto, habrá que enviarle un mensaje preguntándole el precio. El diseño correspondiente aparece en la figura 18.6.

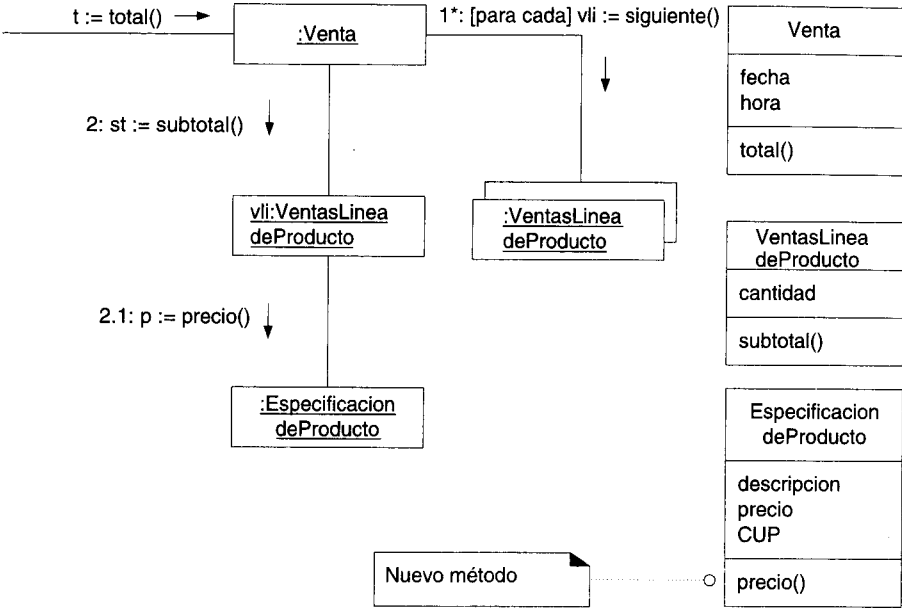


Figura 18.6 Cálculo del total de Venta.

En conclusión, para cumplir con la responsabilidad de conocer y dar el total de la venta, se asignaron tres responsabilidades a las tres clases de objeto así:

Clase	Responsabilidad
Venta	conoce el total de la venta
VentasLineadeProducto	conoce el subtotal de la línea de producto
EspecificaciondeProducto	conoce el precio del producto

El contexto donde las responsabilidades se consideraron y se decidieron fue el dibujo de un diagrama de colaboración. Después, la sección dedicada a métodos en un diagrama de clases puede resumir los métodos.

El principio en virtud del cual se asignaron las responsabilidades fue el patrón Experto: lo colocamos junto con el objeto que posee la información necesaria para cumplirlas.

Explicación Experto es un patrón que se usa más que cualquier otro al asignar responsabilidades; es un principio básico que suele utilizarse en el diseño orientado a objetos. Con él no se pretende designar una idea oscura ni extraña; expresa simplemente la “intuición” de que los objetos hacen cosas relacionadas con la información que poseen.

Nótese que el cumplimiento de una responsabilidad requiere a menudo información distribuida en varias clases de objetos. Ello significa que hay muchos expertos “parciales” que colaboraron en la tarea. Por ejemplo, el problema del total de la venta exigirá finalmente la colaboración de tres clases. Siempre que la información se encuentre esparcida en varios objetos, éstos habrán de interactuar a través de mensajes para compartir el trabajo.

El patrón Experto da origen a diseño donde el objeto de software realiza las operaciones que normalmente se aplican a la cosa real que representa: a esto Peter Coad lo llama estrategia de “Hacerlo yo mismo” [Coad95]. Por ejemplo, en el mundo real una venta no nos indica su total sin ayuda de aparatos electromecánicos; se trata de un concepto inanimado. Alguien calcula el total de la venta. Pero en el terreno del software orientado a objetos, todos los objetos del software están “vivos” o “animados”, y pueden asumir responsabilidades y hacer cosas. Básicamente, hacen cosas relacionadas con la información de que disponen. A esto yo lo llamo principio de “Animación” en el diseño orientado a objetos; es como estar en una caricatura donde todo tiene vida.

El patrón Experto —como tantas otras cosas en la tecnología de objetos— ofrece una analogía con el mundo real. Acostumbramos asignar responsabilidad a individuos que disponen de la información necesaria para llevar a cabo una tarea. Por ejemplo, en una empresa ¿quién será el encargado de preparar un estado de pérdidas y ganancias? El empleado que tiene acceso a toda la información necesaria para elaborarlo: quizá el director financiero. Y del mismo modo que los objetos del software colaboran porque la información está esparcida, lo mismo sucede con el personal de una empresa. El director financiero podrá pedir a los encargados de cuentas por cobrar y de cuentas por pagar que generen reportes individuales sobre créditos y deudas.

- Beneficios**
- Se conserva el encapsulamiento, ya que los objetos se valen de su propia información para hacer lo que se les pide. Esto soporta un **bajo acoplamiento**, lo que favorece al hecho de tener sistemas más robustos y de fácil mantenimiento. (Bajo Acoplamiento es un patrón GRASP que examinaremos más adelante.)
 - El comportamiento se distribuye entre las clases que cuentan con la información requerida, alentando con ello definiciones de clase “sencillas” y más cohesivas que son más fáciles de comprender y de mantener. Así se brinda soporte a una **alta cohesión** (patrón del que nos ocuparemos luego).

Otras formas de designar este patrón “Juntar responsabilidades y la información”, “Lo hace el que conoce”, “Animación”, “Lo hago yo mismo”, “Unir los servicios a los atributos sobre los que operan”.

18.10 Creador

Solución Asignarle a la clase B la responsabilidad de crear una instancia de clase A en uno de los siguientes casos:

- B *agrega* los objetos A.
- B *contiene* los objetos A.
- B *registra* las instancias de los objetos A.
- B *utiliza* específicamente los objetos A.
- B *tiene los datos de inicialización* que serán transmitidos a A cuando este objeto sea creado (así que B es un Experto respecto a la creación de A).

B es un *creador* de los objetos A.

Si existe más de una opción, prefiera la clase B que *agregue* o *contenga* la clase A.

Problema ¿Quién debería ser responsable de crear una nueva instancia de alguna clase?

La creación de objetos es una de las actividades más frecuentes en un sistema orientado a objetos. En consecuencia, conviene contar con un principio general para asignar las responsabilidades concernientes a ella. El diseño, bien asignado, puede soportar un bajo acoplamiento, una mayor claridad, el encapsulamiento y la reutilizabilidad.

Ejemplo En la aplicación del punto de venta, ¿quién debería encargarse de crear una instancia *VentasLineadeProducto*? Desde el punto de vista del patrón Creador, deberíamos buscar una clase que agregue, contenga y realice otras operaciones sobre este tipo de instancias. Examine atentamente el modelo conceptual parcial de la figura 18.7.

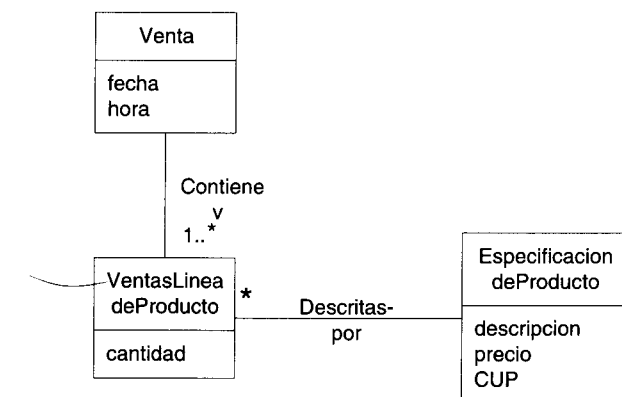


Figura 18.7 Modelo conceptual parcial.

Una *Venta* contiene (en realidad, agrega) muchos objetos *VentasLineadeProducto*; por ello, el patrón Creador sugiere que *Venta* es idónea para asumir la responsabilidad de crear las instancias *VentasLineadeProducto*.

Lo anterior nos permite diseñar las interacciones de los objetos en un diagrama de colaboración, según se observa en la figura 18.8.

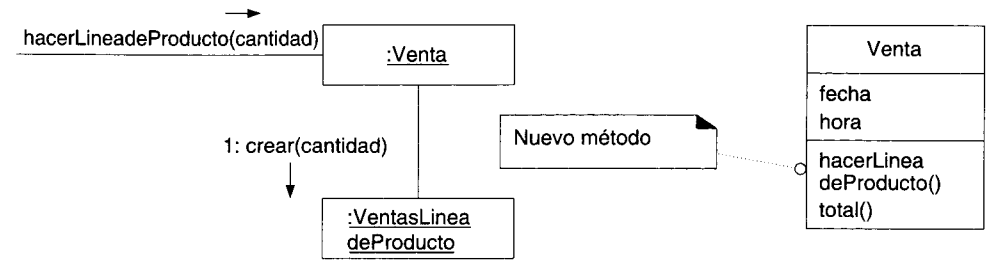


Figura 18.8 Creación de un objeto VentasLineadeProducto.

Esta asignación de responsabilidades requiere definir en *Venta* un método de *hacerLineadeProducto*.

Una vez más, el contexto donde se consideraron y se asignaron estas responsabilidades fue el momento de dibujar un diagrama de colaboración. Después, en la sección destinada a métodos en un diagrama de clases, podemos resumir los resultados de la asignación de responsabilidades, realizadas concretamente como métodos.

Explicación El patrón Creador guía la asignación de responsabilidades relacionadas con la creación de objetos, tarea muy frecuente en los sistemas orientados a objetos. El propósito fundamental de este patrón es encontrar un creador que debemos conectar con el objeto producido en cualquier evento. Al escogerlo como creador, se da soporte al bajo acoplamiento.

El Agregado *agrega* la Parte, el Contenedor *contiene* el contenido, el Registro *registra*. En un diagrama de clases se registran las relaciones muy frecuentes entre las clases. El patrón Creador indica que la clase incluyente del contenedor o registro es idónea para asumir la responsabilidad de crear la cosa contenida o registrada. Desde luego, se trata tan sólo de una directriz.

Nótese que el concepto de **agregación** se utilizó al examinar el patrón Creador. Es un tema que trataremos más ampliamente en un capítulo posterior; pero damos aquí una definición sucinta: la agregación incluye cosas que están en una sólida relación de parte-todo o de parte-estructura; por ejemplo, Cuerpo agrega Pierna y Párrafo agrega oración.

En ocasiones encontramos un patrón creador buscando la clase con los datos de inicialización que serán transferidos durante la creación. Éste es en realidad un ejemplo del patrón Experto. Los datos de inicialización se transmiten durante la creación a través de algún método de inicialización, como un constructor en Java que cuenta con parámetros.

Suponga, por ejemplo, que una instancia *Pago* al momento de ser creada necesita inicializarse con el total *Venta*. Puesto que *Venta* conoce el total, es un buen candidato para ser el parámetro creador de *Pago*.

Beneficios ■ Se brinda soporte a un **bajo acoplamiento** (que describiremos más adelante), lo cual supone menos dependencias respecto al mantenimiento y mejores oportunidades de reutilización. Es probable que el acoplamiento no aumente, pues la clase *creada* tiende a ser visible a la clase *creador*, debido a las asociaciones actuales que nos llevaron a elegirla como el parámetro adecuado.

patrones Conexos ■ Bajo acoplamiento.
■ Parte-todo [BMRSS96] describe un patrón para definir los objetos que soportan el encapsulamiento de sus componentes.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECA
MONTEVIDEO - URUGUAY

18.11 Bajo acoplamiento

Solución Asignar una responsabilidad para mantener bajo acoplamiento.

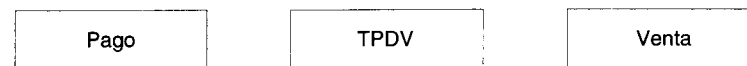
Problema ¿Cómo dar soporte a una dependencia escasa y a un aumento de la reutilización?

El **acoplamiento** es una medida de la fuerza con que una clase está conectada a otras clases, con que las conoce y con que recurre a ellas. Una clase con bajo (o débil) acoplamiento no depende de muchas otras; “muchas otras” depende del contexto, pero no lo estudiaremos aquí por el momento.

Una clase con alto (o fuerte) acoplamiento recurre a muchas otras. Este tipo de clases no es conveniente: presentan los siguientes problemas:

- Los cambios de las clases afines ocasionan cambios locales.
- Son más difíciles de entender cuando están aisladas.
- Son más difíciles de reutilizar porque se requiere la presencia de otras clases de las que dependen.

Ejemplo Examine con mucha atención el siguiente diagrama parcial de clases desde la perspectiva de la aplicación a la terminal del punto de venta:



Suponga que necesitamos crear una instancia *Pago* y asociarla a *Venta*. ¿Qué clase se encargará de hacer esto? Puesto que una instancia *TPDV* “registra” un *Pago* en el dominio del mundo real, el patrón Creador indica que *TPDV* es un buen candidato para producir el *Pago*. La instancia *TPDV* podría entonces enviarle a *Venta* el mensaje *agregarPago*, transmitiendo al mismo tiempo el nuevo *Pago* como parámetro. En la figura 18.9 se incluye un posible diagrama parcial de colaboración que representa gráficamente esto.

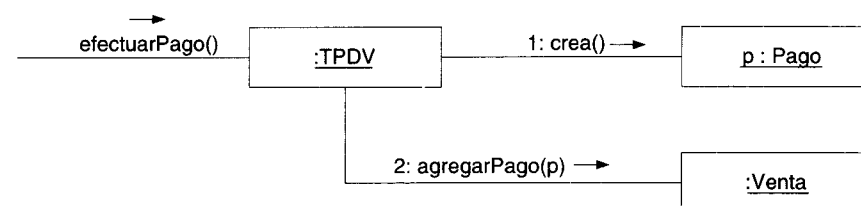


Figura 18.9 TPDV crea Pago.

Esta asignación de responsabilidades acopla la clase *TPDV* al conocimiento de la clase *Pago*. En la figura 18.10 se muestra una solución alterna para crear el *Pago* y asociarlo a la *Venta*.

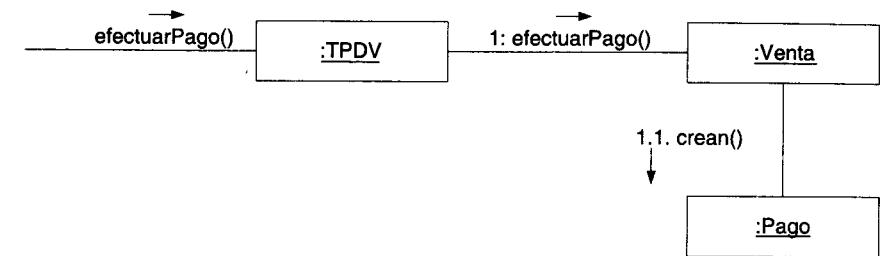


Figura 18.10 Venta crea Pago.

¿Qué diseño, basado en la asignación de responsabilidades, brinda soporte al patrón Bajo Acoplamiento? En ambos casos suponemos que *Venta* terminará acoplándose al conocimiento de un *Pago*. El diseño 1, donde la instancia *TPDV* crea *Pago*, incorpora el acoplamiento de *TPDV* a *Pago*; en cambio, el diseño 2, donde la *Venta* realiza la creación de un *Pago*, no incrementa el acoplamiento. Tomando como única perspectiva la del acoplamiento, el diseño 2 es preferible porque se conserva un menor acoplamiento global. Éste es un ejemplo donde dos patrones —Bajo Acoplamiento y Creador— pueden sugerir soluciones distintas. En la práctica, el grado de acoplamiento no puede considerarse aisladamente de otros principios como Experto y Alta Cohesión. Sin embargo, es un factor a considerar cuando se intente mejorar un diseño.

Explicación El Bajo Acoplamiento es un principio que debemos recordar durante las decisiones de diseño: es la meta principal que es preciso tener presente siempre. Es un *patrón evaluativo* que el diseñador aplica al juzgar sus decisiones de diseño.

En los lenguajes orientados a objetos como C++, Java y Smalltalk, las formas comunes de acoplamiento de *TipoX* a *TipoY* son las siguientes:

- *TipoX* posee un atributo (miembro de datos o variable de instancia) que se refiere a una instancia *TipoY* o al propio *TipoY*.
- *TipoX* tiene un método que a toda costa referencia una instancia de *TipoY* o incluso el propio *TipoY*. Suele incluirse un parámetro o una variable local de *TipoY* o bien el objeto devuelto de un mensaje es una instancia de *TipoY*.
- *TipoX* es una subclase directa o indirecta de *TipoY*.
- *TipoY* es una interfaz y *TipoX* la implementa.

El Bajo Acoplamiento estimula asignar una responsabilidad de modo que su colocación no incremente el acoplamiento tanto que produzca los resultados negativos propios de un alto acoplamiento.

El Bajo Acoplamiento soporta el diseño de clases más independientes, que reducen el impacto de los cambios, y también más reutilizables, que acrecientan la oportunidad de una mayor productividad. No puede considerarse en forma independiente

de otros patrones como Experto o Alta Cohesión, sino que más bien ha de incluirse como uno de los principios del diseño que influyen en la decisión de asignar responsabilidades.

El acoplamiento tal vez no sea tan importante, si no se busca la reutilización. Para apoyar una mejor reutilización de los componentes al hacerlo más independientes, el entero contexto de las metas de reuso ha de tenerse en cuenta antes de intentar reducir al mínimo el acoplamiento. Por ejemplo, a veces se dedica demasiado tiempo a la obtención de componentes reutilizables en futuros proyectos “utópicos”, a pesar de que no se sabe con certeza que se necesitarán. Con ello no queremos decir que se pierde tiempo al tratar de reutilizarlos, sólo que han de atenderse las consideraciones de costo-beneficio.

Una subclase está acoplada firmemente con su superclase. La decisión de derivarla de una superclase ha de ser estudiada con mucho cuidado, pues es una forma muy sólida de acoplamiento. Supongamos, por ejemplo, que es necesario guardar persistentemente los objetos en una base de datos relacional o de objetos. En este caso, está bastante generalizado (aunque no se recomienda) el diseño para crear una superclase abstracta denominada *ObjetoPersistente* del cual se derivan otras clases. Esta subclase tiene la desventaja de acoplar estrechamente objetos del dominio con un servicio en particular y la ventaja de una herencia automática del comportamiento de persistencia, una especie de “matrimonio por conveniencia”. Y esto rara vez es una buena decisión en las relaciones.

No existe una medida absoluta de cuándo el acoplamiento es excesivo. Lo importante es que el diseñador pueda determinar el grado actual de acoplamiento y si surgirán problemas en caso de incrementarlo. En términos generales, han de tener escaso acoplamiento las clases muy genéricas y con grandes probabilidades de reutilización.

El caso extremo de Bajo Acoplamiento ocurre cuando existe poco o nulo acoplamiento entre las clases. Ello no conviene porque una metáfora esencial en la tecnología de objetos es un sistema de objetos conectados que se comunican entre sí a través de mensajes. Si el Bajo Acoplamiento se lleva a los extremos, dará origen a un diseño deficiente por producir objetos incoherentes, atiborrados y complejos que hacen todo el trabajo, con muchos otros objetos muy pasivos y de acoplamiento cero que funcionan como meros depósitos de datos. Un grado moderado de acoplamiento entre las clases es normal y necesario si quiere crearse un sistema orientado a objetos, donde las tareas se realicen por colaboración entre objetos conectados.

- Beneficios**
- no se afectan por cambios de otros componentes
 - fáciles de entender por separado
 - fáciles de reutilizar

18.12 Alta cohesión

Solución Asignar una responsabilidad de modo que la cohesión siga siendo alta.

Problema ¿Cómo mantener la complejidad dentro de límites manejables?

En la perspectiva del diseño orientado a objetos, la **cohesión** (o, más exactamente, la cohesión funcional) es una medida de cuán relacionadas y enfocadas están las responsabilidades de una clase. Una alta cohesión caracteriza a las clases con responsabilidades estrechamente relacionadas que no realicen un trabajo enorme.

Una clase con baja cohesión hace muchas cosas no afines o un trabajo excesivo. No conviene este tipo de clases pues presentan los siguientes problemas:

- son difíciles de comprender
- son difíciles de reutilizar
- son difíciles de conservar
- son delicadas: las afectan constantemente los cambios

Las clases con baja cohesión a menudo representan un alto grado de abstracción o han asumido responsabilidades que deberían haber delegado a otros objetos.

Ejemplo El mismo ejemplo que hemos utilizado en el patrón Bajo Acoplamiento puede analizarse en el caso de Alta Cohesión.

Suponga que necesitamos crear una instancia de *Pago* (en efectivo) y asociarla a la *Venta*. ¿Qué clase será la responsable de hacerlo? Como *TPDV* registra un *Pago* en el dominio del mundo real, el patrón Creador sugiere que *TPDV* es un buen candidato para que genere *Pago*. Entonces la instancia *TPDV* podría enviar a *Venta* un mensaje *agregarPago*, transmitiendo al mismo tiempo el nuevo *Pago* como parámetro, según se muestra en la figura 18.11.

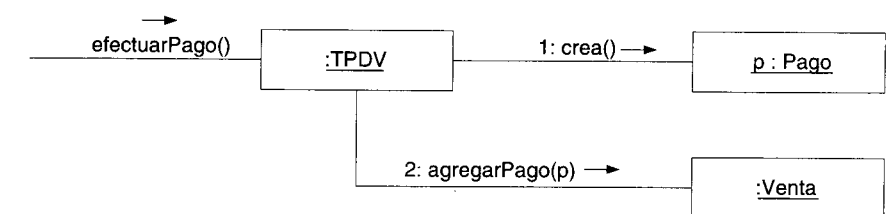


Figura 18.11 TPDV crea Pago.

Esta asignación de responsabilidades coloca en la clase *TPDV* la realización del pago. *TPDV* asume parte de la responsabilidad de realizar la operación del sistema *efectuarPago*.

En este ejemplo aislado, lo anterior es aceptable. Pero si seguimos haciendo que la clase *TPDV* se encargue de efectuar algún trabajo o la mayor parte del que se relaciona con un número creciente de operaciones del sistema, se irá saturando con tareas y terminará por perder la cohesión.

Imagine que el sistema tuviera 50 operaciones, todas ellas recibidas por la clase *TPDV*. Si lleva a cabo el trabajo relacionada con cada una, con el tiempo el sistema será un objeto “saturado” y sin cohesión. Lo importante no es que esta tarea individual de creación de *Pago* haga que *TPDV* pierda su cohesión, sino que sea parte de un panorama más general de la asignación global de responsabilidades y pueda indicar la tendencia a una menor cohesión.

En cambio, como se advierte en la figura 18.12, el segundo diseño delega a la *Venta* la responsabilidad de crear el pago, que soporta una mayor cohesión de *TPDV*. El segundo diseño brinda soporte a una alta cohesión y a un bajo acoplamiento; de ahí que sea conveniente.

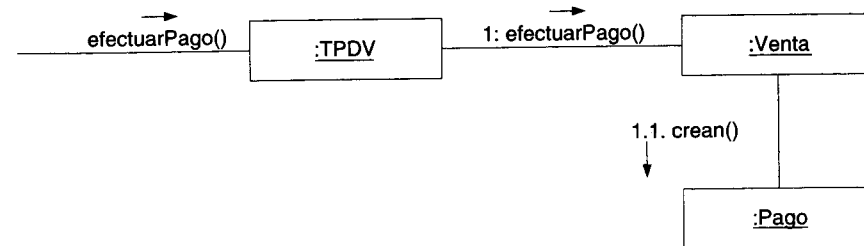


Figura 18.12 Venta crea Pago.

En la práctica, el nivel de cohesión no puede ser considerado independientemente de otras responsabilidades ni de otros principios como los patrones Experto y Bajo Acoplamiento.

Explicación Como el patrón Bajo Acoplamiento, también Alta Cohesión es un principio que debemos tener presente en todas las decisiones de diseño: es la meta principal que ha de buscarse en todo momento. Es un patrón evaluativo que el desarrollador aplica al valorar sus decisiones de diseño.

Grady Booch señala que se da una alta cohesión funcional cuando los elementos de un componente (clase, por ejemplo) “colaboran para producir algún comportamiento bien definido” [Booch94].

A continuación se mencionan algunos escenarios que ejemplifican los diversos grados de la cohesión funcional:

1. *Muy baja cohesión*. Una clase es la única responsable de muchas cosas en áreas funcionales muy heterogéneas.
 - Suponga que existe una clase llamada *RDB-RPC-Interfaz*, la única encargada de interactuar con las bases relacionales de datos y de atender las llamadas de procedimientos remotos. Éstas son dos áreas funcionales radicalmente distintas, y cada una requiere mucho código de soporte. Las responsabilidades deberían repartirse en una familia de clases relacionada con el acceso RDB y en otra familia relacionada con el soporte RPC.

2. *Baja cohesión*. Una clase tiene la responsabilidad exclusiva de una tarea compleja dentro de un área funcional.
 - Suponga que existe una clase llamada *RDBInterfaz*, la única encargada de interactuar con bases relacionales de datos. Los métodos de una clase están todos relacionados, pero hay muchos de ellos y una enorme cantidad de código de soporte; puede haber cientos o miles de métodos. La clase debería dividirse en una familia de clases ligeras que compartan el trabajo para ofrecer el acceso RDB.
3. *Alta cohesión*. Una clase tiene responsabilidades moderadas en un área funcional y colabora con las otras para llevar a cabo las tareas.
 - Suponga que existe una clase denominada *RDBInterfaz*, que se encarga de una parte de la interacción con bases relacionales de datos. Interactúa con una docena de clases afines mediante el acceso RDB para recuperar objetos y guardarlos.
4. *Cohesión moderada*. Una clase tiene un peso ligero y responsabilidades exclusivas en unas cuantas áreas que están relacionadas lógicamente con el concepto de clase, pero no entre ellas.
 - Suponga que existe una clase denominada *Compañía*, la única responsable de: a) conocer a sus empleados b) conocer la información financiera. Estas dos áreas no están estrechamente relacionadas entre ellas, aunque lógicamente lo están con el concepto de compañía. Además, el número total de métodos públicos es pequeño, lo mismo que el código de soporte.

Una regla práctica es la siguiente: una clase de alta cohesión posee un número relativamente pequeño, con una importante funcionalidad relacionada y poco trabajo por hacer. Colabora con otros objetos para compartir el esfuerzo si la tarea es grande.

Una clase con mucha cohesión es útil porque es bastante fácil darle mantenimiento, entenderla y reutilizarla. Su alto grado de funcionalidad, combinada con una reducida cantidad de operaciones, también simplifica el mantenimiento y los mejoramientos. La ventaja que significa una gran funcionalidad también soporta un aumento de la capacidad de reutilización.

El patrón Alta Cohesión —como tantas otras cosas en la tecnología de objetos— presenta semejanzas con el mundo real. Todos sabemos que, si alguien asume demasiadas responsabilidades —sobre todo las que debería delegar—, no será eficiente. Esto se observa en algunos gerentes que no han aprendido a delegar. Muestran baja cohesión; prácticamente ya están “desligados”.

- Beneficios**
- Mejoran la claridad y la facilidad con que se entiende el diseño.
 - Se simplifican el mantenimiento y las mejoras en funcionalidad.
 - A menudo se genera un bajo acoplamiento.
 - La ventaja de una gran funcionalidad soporta una mayor capacidad de reutilización, porque una clase muy cohesiva puede destinarse a un propósito muy específico.

18.13 Controlador

Solución Asignar la responsabilidad del manejo de un mensaje de los eventos de un sistema a una clase que represente una de las siguientes opciones:

- el “sistema” global (*controlador de fachada*).
- la empresa u organización global (*controlador de fachada*).
- algo en el mundo real que es activo (por ejemplo, el papel de una persona) y que pueda participar en la tarea (*controlador de tareas*).
- un manejador artificial de todos los eventos del sistema de un caso de uso, generalmente denominados “Manejador<NombreCasodeUso>” (*controlador de casos de uso*).

Utilice la misma clase de controlador con todos los eventos del sistema en el mismo caso de uso.

Corolario: Nótese que en esta lista no figuran las clases “ventana”, “aplicación”, “vista” ni “documento”. Estas clases *no* deberían ejecutar las tareas asociadas a los eventos del sistema; generalmente las reciben y las delegan al controlador.

Problema ¿Quién debería encargarse de atender un evento del sistema?

Un **evento del sistema** es un evento de alto nivel generado por un actor externo; es un evento de entrada externa. Se asocia a **operaciones del sistema**: las que emite en respuesta a los eventos del sistema. Por ejemplo, cuando un cajero que usa un sistema de terminal en el punto de venta oprime el botón “Terminar Venta”, está generando un evento sistémico que indica que “la venta ha terminado”. De modo similar, cuando un escritor que usa un procesador de palabras pulsa el botón “revisar ortografía”, está produciendo un evento que indica “realizar una revisión ortográfica”.

Un **Controlador** es un objeto de interfaz no destinada al usuario que se encarga de manejar un evento del sistema. Define además el método de su operación.

Ejemplo En la aplicación del punto de venta se dan varias operaciones del sistema, como se advierte en la figura 18.13.

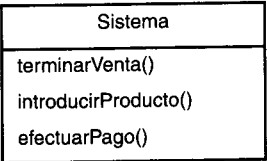


Figura 18.13 Operaciones del sistema asociadas a los eventos sistémicos.

Durante el análisis del comportamiento del sistema, sus operaciones son asignadas al tipo *Sistema*, para indicar que son operaciones del sistema. Pero ello *no* significa que una clase llamada *Sistema* las ejecute durante el diseño.

Más bien, durante el diseño, a la clase Controlador se le asigna la responsabilidad de las operaciones del sistema (figura 18.14).

¿Quién debería ser el controlador de eventos sistémicos como *introducirProducto* y *terminarVenta*?

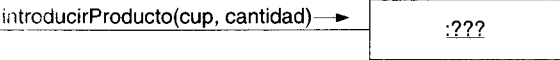
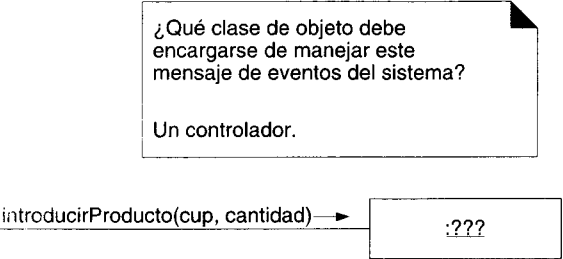


Figura 18.14 ¿Controlador de introducirProducto?

De acuerdo con el patrón Controlador, disponemos de las siguientes opciones:

representa el “sistema” global	<i>TPDV</i>
representa la empresa u organización global	<i>Tienda</i>
representa algo en el mundo real que está activo (por ejemplo, el papel de una persona) y que puede intervenir en la tarea	<i>Cajero</i>
representa un manejador artificial de todas las operaciones del sistema de un caso de uso.	<i>ManejadordeComprar-Productos</i>

Por lo que respecta a los diagramas de colaboración, lo anterior significa que se usará uno de los ejemplos de la figura 18.15.

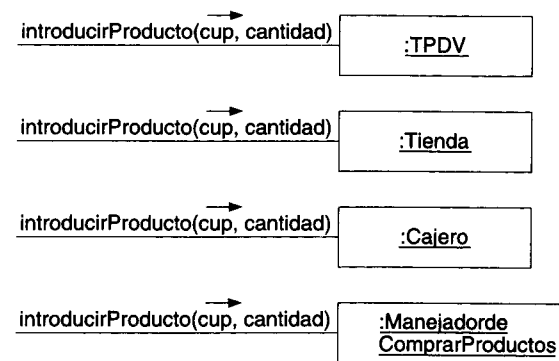


Figura 18.15 Decisiones del controlador.

En la decisión de cuál de las cuatro clases es el controlador más apropiado influyen también otros factores como la cohesión y el acoplamiento. La sección de la explicación profundiza este aspecto.

Durante el diseño, las operaciones del sistema detectadas en el análisis de su comportamiento se asignan a una o más clases de controladores como *TPDV*, según se observa en la figura 18.16.

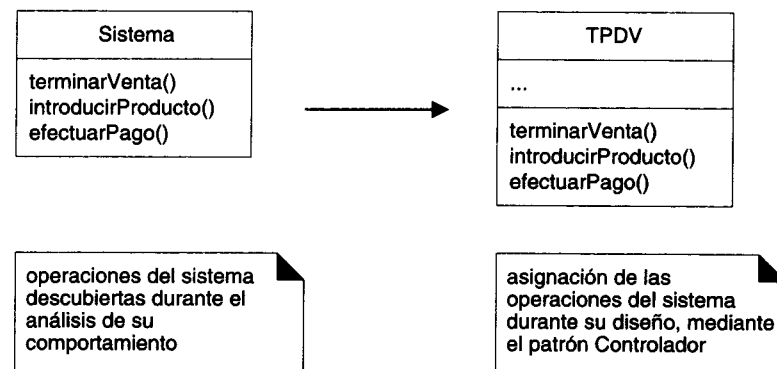


Figura 18.16 Asignación de las operaciones del sistema.

Explicación La mayor parte de los sistemas reciben eventos de entrada externa, los cuales generalmente incluyen una interfaz gráfica para el usuario (IGU) operado por una persona. Otros medios de entrada son los mensajes externos —entre ellos un conmutador de telecomunicaciones para procesar llamadas— o las señales procedentes de sensores como sucede en los sistemas de control de procesos.

En todos los casos, si se recurre a un diseño orientado a objetos, hay que elegir los controladores que manejen esos eventos de entrada. Este patrón ofrece una guía para tomar decisiones apropiadas que generalmente se aceptan.

La misma clase controlador debería utilizarse con todos los eventos sistémicos de un caso de uso, de modo que podamos conservar la información referente al estado del caso. Esta información será útil —por ejemplo— para identificar los eventos del sistema fuera de secuencia (entre ellos, una operación *efectuarPago* antes de *terminarVenta*). Puede emplearse varios controladores en los casos de uso.

Un defecto frecuente al diseñar controladores consiste en asignarles demasiada responsabilidad. Normalmente un controlador debería delegar a otros objetos el trabajo que ha de realizarse mientras coordina la actividad. Favor de consultar la sección *Problemas y soluciones* donde se dan más detalles al respecto.

La primera categoría de controlador es un controlador de fachada que representa al “sistema” global. Es una clase que, para el diseñador, representa de alguna manera al sistema entero. Podría tratarse de una unidad física, como una clase *TPDV*, *Interrup-tordeTelecomunicaciones* o *Robot*; una clase que representa todo el sistema de software, como *SistemadeInformacionAlmenudeo*, *SistemadeManejodeMensajes* o *ControladordeRobot*, o cualquier otro concepto que, por decisión del diseñador, denote el sistema global.

Los controladores de fachada son adecuados cuando el sistema sólo tiene unos cuantos eventos o cuando es imposible redirigir los mensajes de los eventos del sistema a otros controladores, como sucede en un sistema de procesamiento de mensajes.

Si se recurre a la cuarta categoría de controlador —un “manejador artificial de casos de uso”—, habrá entonces un controlador para cada caso. Adviértase que éste no es un objeto del dominio; es un concepto artificial cuyo fin es dar soporte al sistema (una *fabricación pura*, en términos de los patrones de GRASP). Por ejemplo, si la aplicación del punto de venta contiene casos de uso como “Comprar Productos” y “Devolver Productos”, habrá una clase *ManejadordeComprarProductos* y una clase *ManejadordeDevolverProductos*.

¿Cuándo deberíamos escoger un controlador de casos de uso? Es una alternativa que debe tenerse en cuenta si el hecho de asignar las responsabilidades en cualquiera de las otras opciones de controlador genera diseños de baja cohesión o alto acoplamiento. Esto ocurre generalmente cuando un controlador empieza a “saturarse” con demasiadas responsabilidades. Un controlador de casos de uso constituye una buena alternativa cuando hay muchos eventos de sistema entre varios procesos: asigna su manejo a clases individuales controlables, además de que ofrece una base para reflexionar sobre el estado del proceso actual.

Un corolario importante del patrón Controlador es que los objetos externos de conexión (por ejemplo, los objetos ventana, los applets —o pequeñas aplicaciones—) y la capa de presentación no deberían tener la responsabilidad de llevar a cabo los eventos del sistema.

Dicho con otras palabras, las operaciones del sistema —las cuales reflejan los procesos de la empresa o el dominio— deberían manejarse en la capa de dominio de los objetos y no en las de interfaz, presentación o aplicación. Véase un ejemplo en la siguiente sección titulada *Problemas y Soluciones*.

- Beneficios**
- *Mayor potencial de los componentes reutilizables.* Garantiza que la empresa o los procesos de dominio sean manejados por la capa de los objetos del dominio y no por la de la interfaz. Desde el punto de vista técnico, las responsabilidades del controlador podrían cumplirse en un objeto de interfaz, pero esto supone que el código del programa y la lógica relacionada con la realización de los procesos del dominio puro quedarían incrustados en los objetos interfaz o ventana. Un diseño de interfaz-como-controlador reduce la posibilidad de reutilizar la lógica de los procesos del dominio en aplicaciones futuras, por estar ligada a una interfaz determinada (por ejemplo, un objeto similar a una ventana) que rara vez puede utilizarse en otras aplicaciones. En cambio, el hecho de delegar a un controlador la responsabilidad de la operación de un sistema entre las clases del dominio soporta la reutilización de la lógica para manejar los procesos afines del negocio en aplicaciones futuras.
 - *Reflexionar sobre el estado del caso de uso.* A veces es necesario asegurarse de que las operaciones del sistema sigan una secuencia legal o poder razonar sobre el estado actual de la actividad y las operaciones en el caso de uso subyacente. Por ejemplo, tal vez deba garantizarse que la operación *efectuarPago* no ocurra mientras no se concluya la operación *terminarVenta*. De ser así, esta información sobre el estado ha de capturarse en alguna parte; el controlador es una buena opción, sobre todo si se emplea a lo largo de todo el caso (cosa que recomendamos).

Problemas y soluciones **Controladores saturados**

Una clase controlador mal diseñada presentará baja cohesión: está dispersa y tiene demasiadas áreas de responsabilidad; recibe el nombre de **controlador saturado**. Entre los signos de saturación podemos mencionar los siguientes:

- Hay una *sola* clase controlador que recibe *todos* los eventos del sistema y éstos son excesivos. Tal situación suele ocurrir si se escoge un controlador de papeles o de fachada.
- El controlador realiza él mismo muchas tareas necesarias para cumplir el evento del sistema, sin que delegue trabajo. Esto suele constituir una violación de los patrones Experto y Alta Cohesión.
- Un controlador posee muchos atributos y conserva información importante sobre el sistema o dominio —información que debería haber sido redistribuida entre otros objetos— y también duplica la información en otra parte.

Hay varias formas de resolver el problema de un controlador saturado, entre las que se encuentran las siguientes:

1. Agregar más controladores: un sistema no necesariamente debe tener uno solamente. Además de los controladores de fachada, recomendamos emplear los controladores de papeles o los de casos de uso. Considere, por ejemplo, una aplicación con

muchos eventos sistémicos como podría serlo el sistema de reservaciones de una línea aérea. Puede incluir los siguientes controladores:

Controladores de papeles	Controladores de casos de uso
Agente de Reservaciones	Manejador de Hacer Reservación
Programador de Vuelos	Manejador de Administración de Programación
Analista de Tarifas	Manejador de Administración de Tarifas

2. Diseñe el controlador de modo que delegue fundamentalmente a otros objetos el desempeño de las responsabilidades de la operación del sistema.

Advertencia: los controladores de papeles pueden conducir a la obtención de diseños deficientes

Asignar una responsabilidad a un objeto de papeles humanos en una forma que imite lo que ese papel realiza en el mundo real (por ejemplo, el objeto de software *Cajero* que maneja *efectuarPago*) es aceptable, a condición de que el diseñador conozca perfectamente los posibles riesgos y los evite. En particular, se corre el peligro de generar un controlador poco coherente de papeles que no delegue. Un buen diseño orientado a objetos “les da vida”, al asignarles responsabilidades, aunque representen cosas inanimadas del mundo real. Si escoge un controlador de papeles, no caiga en la trampa de diseñar objetos de tipo humano que realicen todo el trabajo; opte más bien por delegar.

En términos generales, aconsejamos usar poco los controladores de papeles.

La capa de presentación no maneja los eventos del sistema

Un corolario importante —lo repetimos— del patrón Controlador es que los objetos de la interfaz (por ejemplo, objetos de ventanas, applets) y la capa de presentación no deberían encargarse de manejar los eventos del sistema. En otras palabras, las operaciones de éste —que reflejan los procesos de la empresa o del dominio— han de realizarse en la capa del dominio de objeto y no en la de interfaz, la de presentación ni la de aplicación.

Un ejemplo es un diseño en Java que se sirve de un applet para mostrar la información.

Suponga que la aplicación del punto de venta tiene una ventana que muestra la información de ventas y que captura las operaciones del cajero. Con un patrón Controlador, la figura 18.17 describe gráficamente una relación aceptable entre el applet, el Controlador y otros objetos de la parte del sistema correspondiente al punto de venta (con simplificaciones).

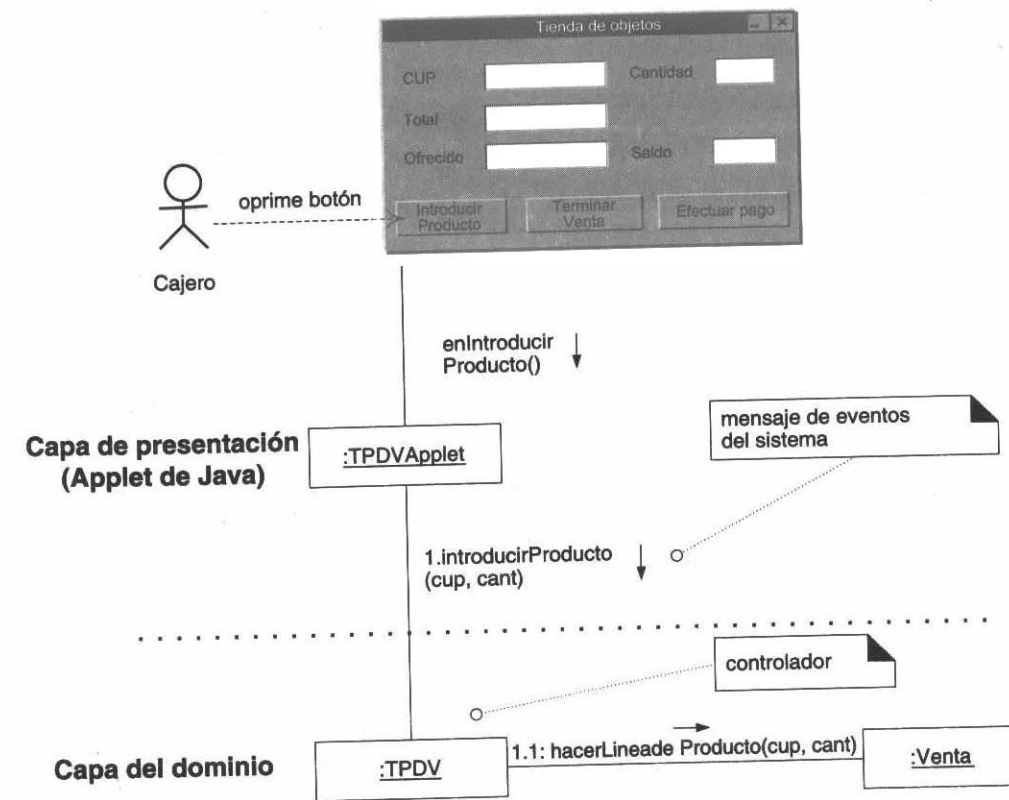


Figura 18.17 Acoplamiento adecuado de la capa de presentación a la capa del dominio.

Nótese que la clase *TPDVApplet* —parte de la capa de presentación— transmite un mensaje *introducirProducto* al objeto *TPDV*. No intervino en el procesamiento de la operación ni la decisión de cómo manejarla, el applet se limitó a delegarla a la capa del dominio.

El potencial de reutilización aumenta al asignar, mediante el patrón Controlador, la responsabilidad de las operaciones del sistema a los objetos situados en la capa del dominio y no en los soportes de la capa de presentación. Si un objeto capa de interfaz (como *TPDVApplet* manejó una operación del sistema —que representa parte de un proceso de negocios—, la lógica de la operación quedará contenida en una interfaz (de tipo ventana, por ejemplo), que tiene pocas posibilidades de reutilización por su acoplamiento con una interfaz y aplicaciones específicas.

En consecuencia, no recomendamos el diseño de la figura 18.18.

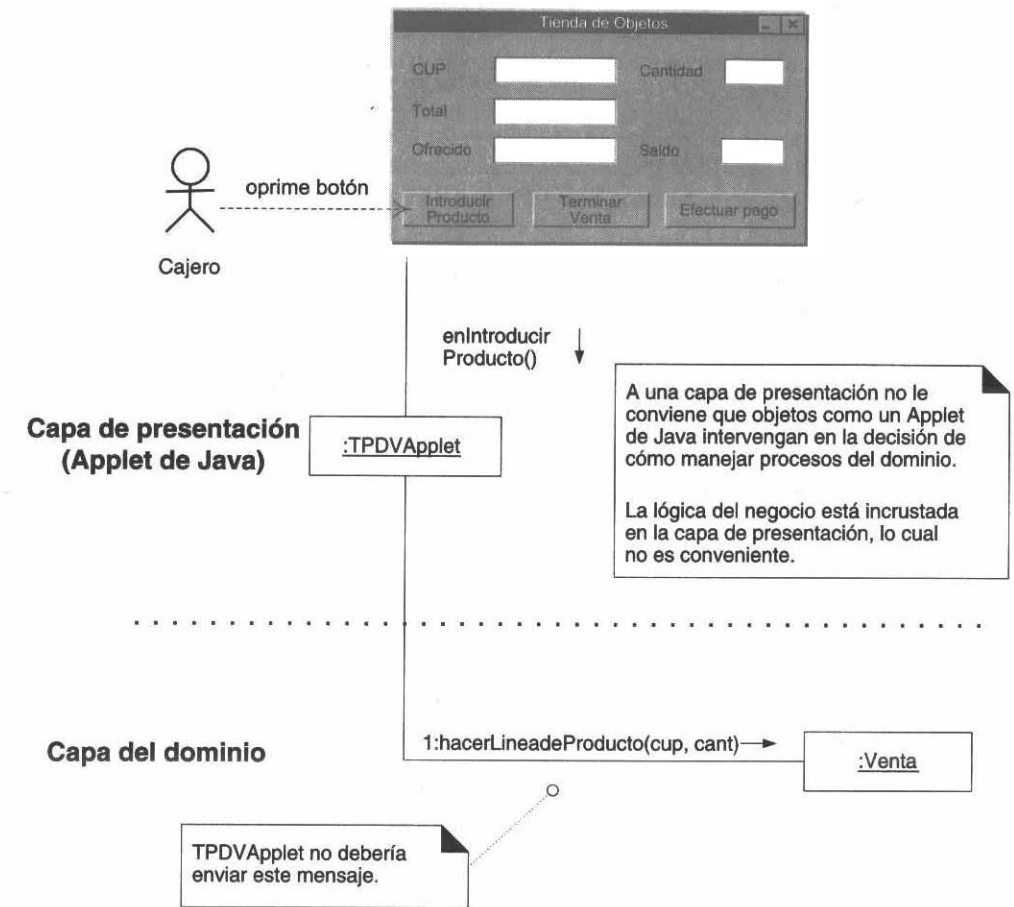


Figura 18.18 Acoplamiento inadecuado de la capa de presentación a la capa del dominio.

Asignar la responsabilidad de la operación del sistema a un controlador del objeto dominio facilita volver a utilizar la lógica del programa que soporte el proceso afín del negocio en aplicaciones futuras. Facilita asimismo desconectar la capa de interfaz y usar otro esquema u otra tecnología de interfaz o bien ejecutar el sistema en un modo “lotes” fuera de línea.

Sistemas del manejo de mensajes y el patrón Comando

Muchas aplicaciones no cuentan con una interfaz para el usuario y, en cambio, reciben mensajes provenientes de algún otro sistema externo; son sistemas de manejo de mensajes. Un conmutador de telecomunicaciones es un ejemplo muy conocido. El mensaje puede estar codificado en un registro, en una secuencia de bytes sin procesar o en un mensaje “real” dirigido a un objeto, si se usa un mecanismo de comunicación interprocesos orientado a objetos como CORBA.

En una aplicación con una interfaz para usuario (una ventana, por ejemplo), la ventana puede decidir cuál será el objeto controlador. Varias ventanas pueden colaborar con los controladores, sobre todo si se recurre a un controlador de casos de uso.

En cambio, el diseño de la interfaz y del controlador es diferente en una aplicación de manejo de mensajes. Tratándose de un caso simple, recomendamos el siguiente diseño. Si el lector desea un diseño más complejo y de muchas características, consulte el patrón Emisor-Receptor en [BMRSS96].

Para manejar los mensajes de eventos del sistema en un sistema de este tipo:

1. Defina un solo controlador para todos los mensajes de eventos; puede ser un controlador de fachada o un controlador individual de casos de uso, con un nombre como *ManejadordeMensajes*.
2. Use el patrón Comando [GHJV95] para contestar la consulta.

El patrón **Comando** especifica la definición de una clase en cada mensaje o comando, todas ellas con el método *ejecutar*. El controlador creará una instancia Comando correspondiente al mensaje del evento del sistema y le enviará un mensaje *ejecutar*. Las clases comando cuentan con un método único *ejecutar* que especifica las acciones de él, como se aprecia en la figura 18.19.¹

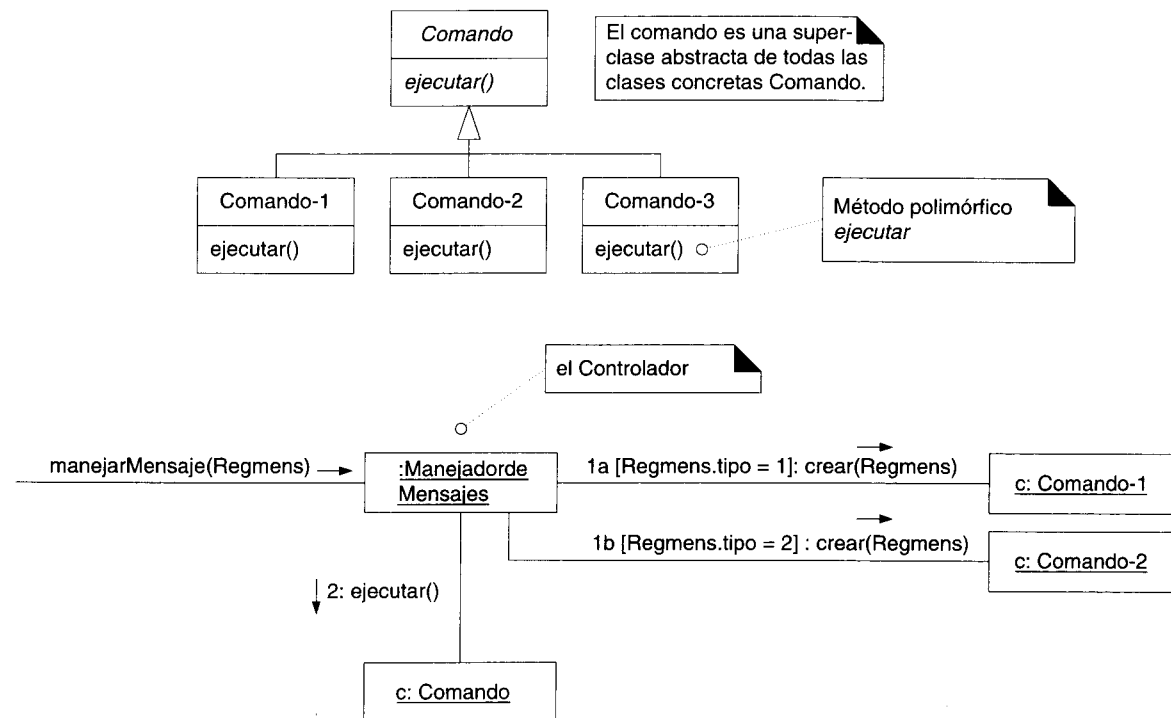


Figura 18.19 El controlador y el uso del patrón Comando.

¹ También es posible un Comando Abstracto *interfaz* en contraste con una superclase abstracta.

Patrones relacionados

- **Comando**: en un sistema de manejo de mensajes; un objeto aislado Comando puede representar y manejar los mensajes [GHJV95].
- **Fachada**: la selección de un objeto que represente un sistema u organización global para que sea controlador lo hace un tipo Fachada [GHJV95].
- **Emisor-Receptor**: éste es un patrón de Siemens [BMRSS96] útil en los sistemas de manejo de mensajes.
- **Capas**: éste es un patrón Siemens [BMRSS96]. Colocar la lógica del dominio en su capa y no en la capa de presentación es parte del patrón Capas.
- **Artefacto puro**: es otro patrón de GRASP. El artefacto puro es una clase artificial, no un concepto del dominio. Un controlador de casos de uso es un tipo de Artefacto Puro.

18.14 Responsabilidades, representación de papeles y las tarjetas CRC

Aunque no sean una parte formal del lenguaje UML, las **tarjetas CRC** [BC89] (siglas en inglés de *Class-Responsibility-Collaborator* Clase-Responsabilidad-Colaborador) son otra herramienta con la cual a veces se asignan las responsabilidades y se indica una colaboración con otros objetos. Fueron inventadas por Kent Beck y Ward Cunningham, principales promotores para que los diseñadores de objetos piensen en términos más abstractos sobre la asignación de responsabilidades y de las colaboraciones.

Las tarjetas CRC son tarjetas de índices, una por cada clase, sobre las cuales se abrevian las responsabilidades de la clase y se anota una lista de los objetos con los que colaboran para desempeñarlas. Suelen elaborarse en una sesión de grupos pequeños donde los participantes **representan papeles** de las diversas clases. Cada grupo tiene las tarjetas CRC correspondientes a las clases cuyo papel está desempeñando.

La representación de papeles hace divertido aprender a pensar en función de objetos y de responsabilidades. Pero las tarjetas basadas en texto tienen una capacidad limitada para registrar las colaboraciones en forma exhaustiva. Los diagramas de colaboración y los de clase describen mejor las conexiones entre los objetos y el contexto general.

Recomendamos por su utilidad las actividades grupales de diseño dirigidas a asignar responsabilidades y a representar papeles; los resultados pueden anotarse después en las tarjetas CRC o en diagramas. Las tarjetas son un método de registrar los resultados de la asignación de responsabilidades y de las colaboraciones. El registro es más satisfactorio si se utiliza este tipo de diagramas. El valor real no está constituido por las tarjetas, sino por el análisis de la asignación de responsabilidades y por la actividad relacionada con la representación de papeles.

DISEÑO DE UNA SOLUCIÓN CON OBJETOS Y PATRONES

Objetivos

- Aplicar los patrones GRASP para asignar responsabilidades a las clases.
- Utilizar la notación de UML en los diagramas de colaboración para describir gráficamente el diseño de la interacción de objetos.

19.1 Introducción

En este capítulo explicaremos cómo crear los diagramas de interacción (en este caso, diagramas de colaboración) para la aplicación del punto de venta. Estudiaremos también su relación con los artefactos anteriores, entre ellos los contratos y el modelo conceptual. Prestamos especial atención a la aplicación de los patrones GRASP para alcanzar una solución bien diseñada.

En el presente capítulo se exponen los principios, mediante el ejemplo del punto de venta, en virtud de los cuales un diseñador orientado a objetos asigna las responsabilidades y establece las interacciones de ellos, una habilidad fundamental en el desarrollo orientado a objetos.

En la práctica, los diseñadores se percatan de que la preparación de los diagramas de interacción es uno de los pasos más lentos (y redituables). Adviértase además lo siguiente:

La asignación de responsabilidades y la elaboración de los diagramas de interacción representan uno de los pasos más importantes en la fase del diseño.